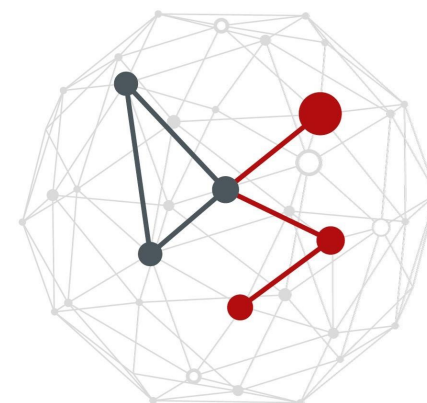


AUTOENCODERS

Michele Rossi

michele.rossi@dei.unipd.it

Dept. of Information Engineering
University of Padova, IT



Outline



- Why? What? How?
- **Review of some matrix algebra**
 - Singular Value Decomposition (SVD)
 - Frobenious norm - def and main properties
 - **Revisiting PCA**: minimum error vs maximum variance formulations
- **Unsupervised learning** with FFNN
- **Autoencoders (AE)**
 - Definition, training objective
 - Equivalence between linear AE and PCA
 - Nonlinear AE
 - Learning strategies
- **Denoising AE**
- **Application example**
 - The ECG signal case

Why? What? How?

- What we are going to see in this lesson
 - We are interested in (for now) **i.i.d. data sequences**
- Data points (samples)
 - Are generated one at a time
 - Can be either **i.i.d.** or **time correlated**
 - Are **sequentially fed** to an algorithm
 - To capture some key features of the data
- Learning objectives
 - **i.i.d. seqs**: meaningful and compact descriptors (feature vectors)
 - Correlated seqs: capture temporal evolution (RNN)
 - Will be treated in another lesson

Unsupervised learning

- We are interested in
 - Unsupervised learning algorithms
 - That automatically extract useful structure from data
- Useful to what?
 - To (i) compress, (ii) classify, (ii) predict (or interpolate)

“We expect unsupervised learning to become *far more important in the longer term*. Human and animal learning is largely unsupervised: we discover the structure of the world by observing it, not by being told the name of every object.” [LeCun15]

[LeCun15] Yann LeCun, Yoshua Bengio, Geoffrey Hinton, “Deep Learning,” *Nature*, 2015.

MATRIX ALGEBRA: LOW-RANK APPROXIMATIONS

Singular Value Decomposition (SVD)

- It is a factorization of a real or complex matrix $\mathbf{M}$
- It is a generalization of *eigenvalue decomposition* (which holds for a square matrix)

SVD decomposition:

$$\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\dagger \quad (1)$$

$(\cdot)^\dagger$ means transpose conjugate, if $\mathbf{M}$ is a real matrix, the same relation holds with all matrices real and using the transpose

Singular Value Decomposition (SVD)

SVD Theorem: Let $\mathbf{M}$ be a complex $m \times n$ matrix with

$$\text{rank}(\mathbf{M}) = r \leq \min(m, n)$$

Then $\mathbf{M}$ can be factorized as

$$\mathbf{M} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\dagger$$

- $\mathbf{\Sigma}$ is an $m \times n$ *diagonal* matrix with *non-negative* real numbers σ_i **on the diagonal**, called the *singular values* of $\mathbf{M}$

$$\sigma_i = \Sigma_{ii} = \sqrt{\lambda_i} \geq 0, \quad i = 1, \dots, m$$

- The number of non-zero (and non-negative) singular values is r
- λ_i are the eigenvalues of $\mathbf{M}^\dagger \mathbf{M}$

Singular Value Decomposition (SVD)

SVD Theorem: Let $\mathbf{M}$ be a complex $m \times n$ matrix with

$$\text{rank}(\mathbf{M}) = r \leq \min(m, n)$$

Then $\mathbf{M}$ can be factorized as

$$\mathbf{M} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\dagger$$

matrix $\mathbf{U}$

- $\mathbf{U}$ is an $m \times m$ complex *unitary matrix*

$$\mathbf{U}^\dagger \mathbf{U} = \mathbf{U} \mathbf{U}^\dagger = \mathbf{U} \mathbf{U}^{-1} = \mathbf{I}$$

- Its columns are called the left-singular vectors of $\mathbf{M}$
- They form an **orthonormal** basis $\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_m$
- They are the eigenvectors of $\mathbf{M} \mathbf{M}^\dagger$

Singular Value Decomposition (SVD)

SVD Theorem: Let $\mathbf{M}$ be a complex $m \times n$ matrix with

$$\text{rank}(\mathbf{M}) = r \leq \min(m, n)$$

Then $\mathbf{M}$ can be factorized as

$$\mathbf{M} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\dagger$$

matrix $\mathbf{V}$

- $\mathbf{V}$ is an $n \times n$ complex *unitary matrix* $\mathbf{V}^\dagger \mathbf{V} = \mathbf{I}$
- Its columns are called the right-singular vectors of $\mathbf{M}$
- They form an *orthonormal* basis $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n$
- They are the eigenvectors of $\mathbf{M}^\dagger \mathbf{M}$

Frobenius norm

- For matrix $\mathbf{M}$ real, we define (see [Appendix 1](#))

$$\|\mathbf{M}\|_F \triangleq \sqrt{\sum_{i,j} |M_{ij}|^2} \quad (2)$$

- if $\mathbf{r}_i$ and $\mathbf{c}_j$ are respectively the rows and columns of $\mathbf{M}$
- It holds

$$\|\mathbf{M}\|_F^2 = \sum_i \|\mathbf{r}_i\|^2 = \sum_j \|\mathbf{c}_j\|^2 \quad (3)$$

- using norm-2 of a vector

$$\mathbf{x} = [x_1, x_2, \dots, x_n]^T \rightarrow \|\mathbf{x}\|^2 \triangleq \sum_{i=1}^n x_i^2$$

Frobenius norm and matrix trace

- For matrix $\mathbf{M}$ real, we define

$$\|\mathbf{M}\|_F \triangleq \sqrt{\sum_{i,j} |M_{ij}|^2}$$

- if $\mathbf{r}_i$ and $\mathbf{c}_i$ are respectively the rows and columns of $\mathbf{M}$
- Consider $\mathbf{M}^T \mathbf{M}$
 - On the main diagonal of this product, we have: $\mathbf{c}_i^T \mathbf{c}_i = \|\mathbf{c}_i\|^2$
 - It follows that

$$\|\mathbf{M}\|_F^2 = \sum_i \|\mathbf{c}_i\|^2 = \text{trace}(\mathbf{M}^T \mathbf{M}) = \text{trace}(\mathbf{M} \mathbf{M}^T) \quad (4)$$

Element-wise inner product

- Let $\mathbf{X}$ and $\mathbf{Y}$ be two matrices
- Let $\mathbf{x}_i$ be column i of $\mathbf{X}$, $\mathbf{y}_j^T$ be row j of $\mathbf{Y}$

$$\langle \mathbf{X}, \mathbf{Y} \rangle_e \triangleq \sum_{i,j} x_{ij} y_{ij} \quad \text{"e" = element-wise}$$

- Given this, it holds

$$\|\mathbf{X}\|_F^2 = \langle \mathbf{X}, \mathbf{X} \rangle_e = \sum_{i,j} x_{i,j}^2$$

- Moreover, it holds

$$\mathbf{XY} = \sum_i \mathbf{x}_i \mathbf{y}_i^T \quad \|\mathbf{XY}\|_F^2 = \langle \mathbf{XY}, \mathbf{XY} \rangle_e$$

col_i(X) x row_i(Y)

Another property

- **Example:** two matrices **A** and **B** (e.g., 2x2), with:

$$\mathbf{A} = \mathbf{xy}^T = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \begin{bmatrix} y_1 & y_2 \end{bmatrix} = \begin{bmatrix} x_1 y_1 & x_1 y_2 \\ x_2 y_1 & x_2 y_2 \end{bmatrix}$$

$$\mathbf{B} = \mathbf{uv}^T = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \begin{bmatrix} v_1 & v_2 \end{bmatrix} = \begin{bmatrix} u_1 v_1 & u_1 v_2 \\ u_2 v_1 & u_2 v_2 \end{bmatrix}$$

$$\langle \mathbf{A}, \mathbf{B} \rangle_e = x_1 y_1 u_1 v_1 + x_1 y_2 u_1 v_2 + x_2 y_1 u_2 v_1 + x_2 y_2 u_2 v_2$$

Another property

- From the following expression, we can collect $y_i v_i$

$$\begin{aligned}\langle \mathbf{A}, \mathbf{B} \rangle_e &= x_1 y_1 u_1 v_1 + x_1 y_2 u_1 v_2 + x_2 y_1 u_2 v_1 + x_2 y_2 u_2 v_2 = \\ &= (x_1 u_1 + x_2 u_2)(y_1 v_1 + y_2 v_2) = \langle \mathbf{x}, \mathbf{u} \rangle (y_1 v_1 + y_2 v_2)\end{aligned}$$

and rewrite:

$$\begin{aligned}\langle \mathbf{x}, \mathbf{u} \rangle \sum_i y_i v_i &= \langle \mathbf{x}, \mathbf{u} \rangle \langle \mathbf{y}, \mathbf{v} \rangle = \\ &= (x_1 u_1 + x_2 u_2)(y_1 v_1 + y_2 v_2) = \\ &= \langle \mathbf{A}, \mathbf{B} \rangle_e = \langle \mathbf{x} \mathbf{y}^T, \mathbf{u} \mathbf{v}^T \rangle_e\end{aligned}$$



$$\langle \mathbf{x} \mathbf{y}^T, \mathbf{u} \mathbf{v}^T \rangle_e = \langle \mathbf{x}, \mathbf{u} \rangle \langle \mathbf{y}, \mathbf{v} \rangle$$

holds in general

Frobenius norm of product of matrices

- Let $\mathbf{X}$ and $\mathbf{Y}$ be two matrices
- Let $\mathbf{x}_i$ be column i of $\mathbf{X}$, $\mathbf{y}_j^T$ be row j of $\mathbf{Y}$

$$\|\mathbf{XY}\|_F^2 = \langle \mathbf{XY}, \mathbf{XY} \rangle_e = \left\langle \sum_i \mathbf{x}_i \mathbf{y}_i^T, \sum_j \mathbf{x}_j \mathbf{y}_j^T \right\rangle_e$$

$$= \sum_{i,j} \langle \mathbf{x}_i, \mathbf{x}_j \rangle \langle \mathbf{y}_i, \mathbf{y}_j \rangle$$

applying property in the previous slide

$$= \sum_i \|\mathbf{x}_i\|^2 \|\mathbf{y}_i\|^2 + \sum_{i \neq j} \langle \mathbf{x}_i, \mathbf{x}_j \rangle \langle \mathbf{y}_i, \mathbf{y}_j \rangle$$

Orthonormal matrices

- We have obtained that

$$\|\mathbf{XY}\|_F^2 = \sum_i \|\mathbf{x}_i\|^2 \|\mathbf{y}_i\|^2 + \sum_{i \neq j} \langle \mathbf{x}_i, \mathbf{x}_j \rangle \langle \mathbf{y}_i, \mathbf{y}_j \rangle$$

- If, e.g., $\mathbf{Y}$ is an orthonormal matrix, it holds

$$\begin{cases} \|\mathbf{y}_i\| = 1 & \forall i \\ \langle \mathbf{y}_i, \mathbf{y}_j \rangle = 0 & i \neq j \end{cases}$$

- and thus

$$\|\mathbf{XY}\|_F^2 = \sum_i \|\mathbf{x}_i\|^2 = \|\mathbf{X}\|_F^2 \quad (5)$$

the same result
holds if $\mathbf{X}$ is
orthonormal

Bound for Frobenius norm of $\mathbf{XY}$

- Another useful result:

plain matrix product

from Cauchy-Schwarz inequality

$$\begin{aligned}\|\mathbf{XY}\|_F^2 &= \sum_i \sum_j \left| \sum_k X_{ik} Y_{kj} \right|^2 \leq \sum_i \sum_j \left(\sum_k |X_{ik}|^2 \sum_k |Y_{kj}|^2 \right) = \\ &= \sum_i \sum_j \left(\sum_{k,\ell} |X_{ik}|^2 |Y_{\ell j}|^2 \right) = \sum_{i,k} |X_{ik}|^2 \sum_{\ell,j} |Y_{\ell j}|^2 = \\ &= \|\mathbf{X}\|_F^2 \|\mathbf{Y}\|_F^2\end{aligned}$$

$$\|\mathbf{XY}\|_F \leq \|\mathbf{X}\|_F \|\mathbf{Y}\|_F \quad (6)$$

Frobenious norm

- Consider generic matrix $\mathbf{M}$ real
- From **SVD** it holds

$$\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \rightarrow \mathbf{M}\mathbf{V} = \mathbf{U}\mathbf{\Sigma}$$

- Which means that (using Eq. (5))

apply F-norm to both sides $\|\mathbf{M}\mathbf{V}\|_F^2 = \|\mathbf{U}\mathbf{\Sigma}\|_F^2 \quad (7)$

- Both $\mathbf{U}$ and $\mathbf{V}$ are *orthonormal*, hence it follows

$$\|\mathbf{M}\|_F^2 = \|\mathbf{\Sigma}\|_F^2 = \sum_{i,j} |\Sigma_{ij}|^2 = \sum_i \sigma_i^2 \quad (8)$$

singular values

Lower rank approximations

- Assume that $\mathbf{M}$ is an $m \times n$ real matrix of rank r , with singular values

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r$$

- And with singular value decomposition $\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$
- Find the best approximation for $\mathbf{M}$, among all real matrices $\mathbf{X}_k$ of size $m \times n$ of lower rank

$$\text{rank}(\mathbf{X}_k) = k \leq r$$

- The best approximation means

$$\|\mathbf{M} - \mathbf{X}_k\|_F = \min_{\mathbf{X} \in \mathcal{M}_{m,n}} \{\|\mathbf{M} - \mathbf{X}\|_F \text{ s.t. } \text{rank}(\mathbf{X}) = k\} \quad (9)$$

Lower rank approximations

- Assume that $\mathbf{M}$ is an $m \times n$ real matrix of rank r , with **singular values**

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r$$

- And with **singular value decomposition** $\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$

- Then, among all real matrices $\mathbf{X}_k$ of size $m \times n$ **of lower rank**

$$\text{rank}(\mathbf{X}_k) = k \leq r$$

Theorem

- The **best low-rank approximation** is $\mathbf{X}_k = \mathbf{U}\mathbf{\Sigma}_k\mathbf{V}^T$ (10)
- Where $\mathbf{\Sigma}_k$ is a *diagonal matrix* with singular values

$$\sigma_1, \sigma_2, \dots, \sigma_k$$

Lower rank approximations

Proof.

- Take generic matrix $\mathbf{X}$ of rank k (by assumption, with $k \leq r$), size $m \times n$
- Writing the Frobenious norm and left- and right-multiplying inside of it by $\mathbf{U}^T$ and $\mathbf{V}$ (the F-norm is invariant), respectively, we obtain

$$\|\mathbf{M} - \mathbf{X}\|_F = \|\mathbf{U}\Sigma\mathbf{V}^T - \mathbf{X}\|_F = \|\Sigma - \mathbf{U}^T\mathbf{X}\mathbf{V}\|_F$$

- Denoting $\mathbf{N} = \mathbf{U}^T\mathbf{X}\mathbf{V}$, an $m \times n$ matrix of rank k , we write:

$$\|\Sigma - \mathbf{N}\|_F^2 = \sum_{i,j} |\Sigma_{ij} - N_{ij}|^2 = \sum_{i=1}^r |\sigma_i - N_{ii}|^2 + \sum_{i>r} |N_{ii}|^2 + \sum_{i \neq j} |N_{ij}|^2$$

1. diagonal terms up to r 2. Diagonal terms after r

↑
due to structure of Σ off-diagonal terms

Lower rank approximations

Proof.

- Up to now, we have found:

$$\begin{aligned}\|\mathbf{M} - \mathbf{X}\|_F^2 &= \|\mathbf{\Sigma} - \mathbf{N}\|_F^2 = \sum_{i,j} |\Sigma_{ij} - N_{ij}|^2 = \\ &= \sum_{i=1}^r |\sigma_i \downarrow - N_{ii}|^2 + \sum_{i>r} |N_{ii}|^2 + \sum_{i \neq j} |N_{ij}|^2\end{aligned}$$

- that **is minimal** if second and third term are **zero**, and first term is **minimized**, i.e.,
$$\begin{cases} N_{ii} = \sigma_i & i = 1, \dots, k \ (k \leq r) \\ N_{ii} = 0 & i > k \\ N_{ij} = 0 & i \neq j \end{cases} \quad (11)$$

Lower rank approximations

Discussion

$$\|\mathbf{M} - \mathbf{X}\|_F^2 = \sum_{i=1}^r |\sigma_i - N_{ii}|^2 + \sum_{i>r} |N_{ii}|^2 + \sum_{i \neq j} |N_{ij}|^2$$

- Is this really the best thing we could do for the first term?
- Maybe, can we make it equal to zero as well?
- By taking

$$\begin{cases} N_{ii} = \sigma_i & i = 1, \dots, r \\ N_{ii} = 0 & i > r \\ N_{ij} = 0 & i \neq j \end{cases}$$

NO, in this case matrix **N** would have rank $r \rightarrow$ NOT permitted, we are looking for a low-rank k approx.

Lower rank approximations

Proof. (continued)

- Using $\mathbf{X}_k$ as in the theorem statement, $\mathbf{X}_k = \mathbf{U}\Sigma_k\mathbf{V}^T$
- From the definition of $\mathbf{N}$, we get

$$\mathbf{N} = \mathbf{U}^T \mathbf{X}_k \mathbf{V} = \mathbf{U}^T \mathbf{U} \Sigma_k \mathbf{V}^T \mathbf{V} = \Sigma_k$$

- Which proves that the matrix that **minimizes the F-norm** is exactly equal to the rank k approx. provided by $\mathbf{X}_k$
- It holds

$$\begin{cases} (\Sigma_k)_{ii} = N_{ii} = \sigma_i & i = 1, \dots, k \\ (\Sigma_k)_{ii} = N_{ii} = 0 & i > k \\ (\Sigma_k)_{ij} = N_{ij} = 0 & i \neq j \end{cases}$$

QED

Lower rank approximations

- Discussion

- Using $\mathbf{X}_k$
- Quantify the **F-norm** of the difference between
 - $\mathbf{M}$: original matrix and $\mathbf{X}_k$: its **low rank approximation**
- From the previous calculations, it descends – **approximation error**

$$\|\mathbf{M} - \mathbf{X}\|_F^2 = \|\mathbf{\Sigma} - \mathbf{N}\|_F^2 = \sum_{i=k+1}^r |\sigma_i|^2$$

- Are the terms not contained in the $\mathbf{N}$ matrix
- Moreover, since $\sigma_i = \sqrt{\lambda_i}$ (λ_i are the eigenvalues of $\mathbf{M}^T \mathbf{M}$)
- It also holds

$$\|\mathbf{M} - \mathbf{X}\|_F^2 = \sum_{i=k+1}^r \lambda_i$$

Divide by n and you get
the average distortion
provided by PCA

PCA – minimum error formulation

- Setup
 - Let $\mathbf{X}$ be the $m \times n$ data matrix
 - And $\mathbf{X}'$ be the zero mean data matrix
 - Obtained by removing the mean vector from all vectors in $\mathbf{X}$

PCA – minimum error formulation

- Setup

- Let $\mathbf{P}$ be the PCA transform matrix $\mathbf{Y}' = \mathbf{P}\mathbf{X}'$
- We define $\mathbf{P}$ by stacking the first $p < m$ eigenvectors (in the rows of $\mathbf{P}$), related to the p largest eigenvalues of:

$$\text{Cov}(\mathbf{X}') = \frac{1}{n} \mathbf{X}'(\mathbf{X}')^T$$

- Since the number of rows of matrix $\mathbf{P}$ is $p < m \rightarrow$ information is lost
- The approximated (reconstructed) data from $\mathbf{Y}'$ is obtained as

$$\tilde{\mathbf{X}}' = \mathbf{P}^T \mathbf{Y}' = \mathbf{P}^T \mathbf{P} \mathbf{X}' \quad (12)$$

- Now, define $\mathbf{W}^T \triangleq \mathbf{P} \in \mathbb{R}^{p \times m}$
- Given all this, the reconstruction error J is

$$J = \|\mathbf{X}' - \tilde{\mathbf{X}}'\|_F^2 = \|\mathbf{X}' - \mathbf{W}\mathbf{W}^T \mathbf{X}'\|_F^2 \quad (13)$$

PCA – minimum error formulation

- **Reconstruction error J** (using (4))

$$\begin{aligned} J &= \|\mathbf{X}' - \mathbf{W}\mathbf{W}^T \mathbf{X}'\|_F^2 = \text{trace}((\mathbf{X}' - \mathbf{W}\mathbf{W}^T \mathbf{X}')(\mathbf{X}' - \mathbf{W}\mathbf{W}^T \mathbf{X}')^T) = \\ &= \text{trace}((\mathbf{X}' - \mathbf{W}\mathbf{W}^T \mathbf{X}')((\mathbf{X}')^T - (\mathbf{X}')^T \mathbf{W}\mathbf{W}^T)) = \\ &= \text{trace}(\mathbf{X}'(\mathbf{X}')^T) - 2\text{trace}(\mathbf{X}'(\mathbf{X}')^T \mathbf{W}\mathbf{W}^T) + \text{trace}(\mathbf{W}\mathbf{W}^T \mathbf{X}'(\mathbf{X}')^T \mathbf{W}\mathbf{W}^T) \end{aligned}$$

using 1) $(\mathbf{A} + \mathbf{B})^T = \mathbf{A}^T + \mathbf{B}^T$ 2) $\text{tr}(\sum_i \mathbf{A}_i) = \sum \text{tr}(\mathbf{A}_i)$ 3) cyclic prop. $\text{tr}(\mathbf{ABC}) = \text{tr}(\mathbf{CAB}) = \text{tr}(\mathbf{BCA})$

- Moreover,

$$\text{trace}(\mathbf{X}'(\mathbf{X}')^T \mathbf{W}\mathbf{W}^T) = \text{trace}((\mathbf{X}')^T \mathbf{W}\mathbf{W}^T \mathbf{X}') \quad \text{cyclic prop. } \text{tr}(\mathbf{ABC}) = \text{tr}(\mathbf{CAB}) = \text{tr}(\mathbf{BCA})$$

$$\text{trace}(\mathbf{W}\mathbf{W}^T \mathbf{X}'(\mathbf{X}')^T \mathbf{W}\mathbf{W}^T) = \text{trace}((\mathbf{X}')^T \mathbf{W}\mathbf{W}^T \mathbf{W}\mathbf{W}^T \mathbf{X}') = \text{trace}((\mathbf{X}')^T \mathbf{W}\mathbf{W}^T \mathbf{X}')$$

cyclic property

$\mathbf{W}^T \mathbf{W} = \mathbf{I}_p$ (orthonormality of eigenvectors)



$$\|\mathbf{X}' - \mathbf{W}\mathbf{W}^T \mathbf{X}'\|_F^2 = \text{trace}(\mathbf{X}'(\mathbf{X}')^T) - \text{trace}(\mathbf{W}^T \mathbf{X}'(\mathbf{X}')^T \mathbf{W})$$

reconstruction error

constant
(does not depend on $\mathbf{W}$)

projected variance
(cyclic prop. again)

Minimum error vs projected variance

$$J = \|\mathbf{X}' - \mathbf{W}\mathbf{W}^T\mathbf{X}'\|_F^2 = \text{trace}(\mathbf{X}'(\mathbf{X}')^T) - \text{trace}(\mathbf{W}^T\mathbf{X}'(\mathbf{X}')^T\mathbf{W}) \quad (14)$$

- This relation says that minimizing the **reconstruction error J** (F-norm, optimization variable is $\mathbf{W}^T$) **is equivalent to maximizing the projected variance** (second, negative term on the right)
- **From the low-rank approx. theorem** $\rightarrow$ the PCA transformation matrix $\mathbf{P}=\mathbf{W}^T$ **minimizes J** and, at the same time, **maximizes the projected variance**
- **Note:** if $p=m$
 - $\mathbf{W}$ is square, invertible, $\mathbf{W}\mathbf{W}^T = \mathbf{I}_m$ and $J=0$

PCA formulation with minimum error

- The PCA transform $\mathbf{W}^T \triangleq \mathbf{P} \in \mathbb{R}^{p \times m}$
- where $\mathbf{Y}' = \mathbf{P}\mathbf{X}'$

is also a solution to:

$$\min_{\mathbf{W} \in \mathbb{R}^{m \times p}} \|\mathbf{X}' - \mathbf{W}\mathbf{W}^T \mathbf{X}'\|_F^2, \text{ subject to: } \mathbf{W}^T \mathbf{W} = \mathbf{I}_p$$

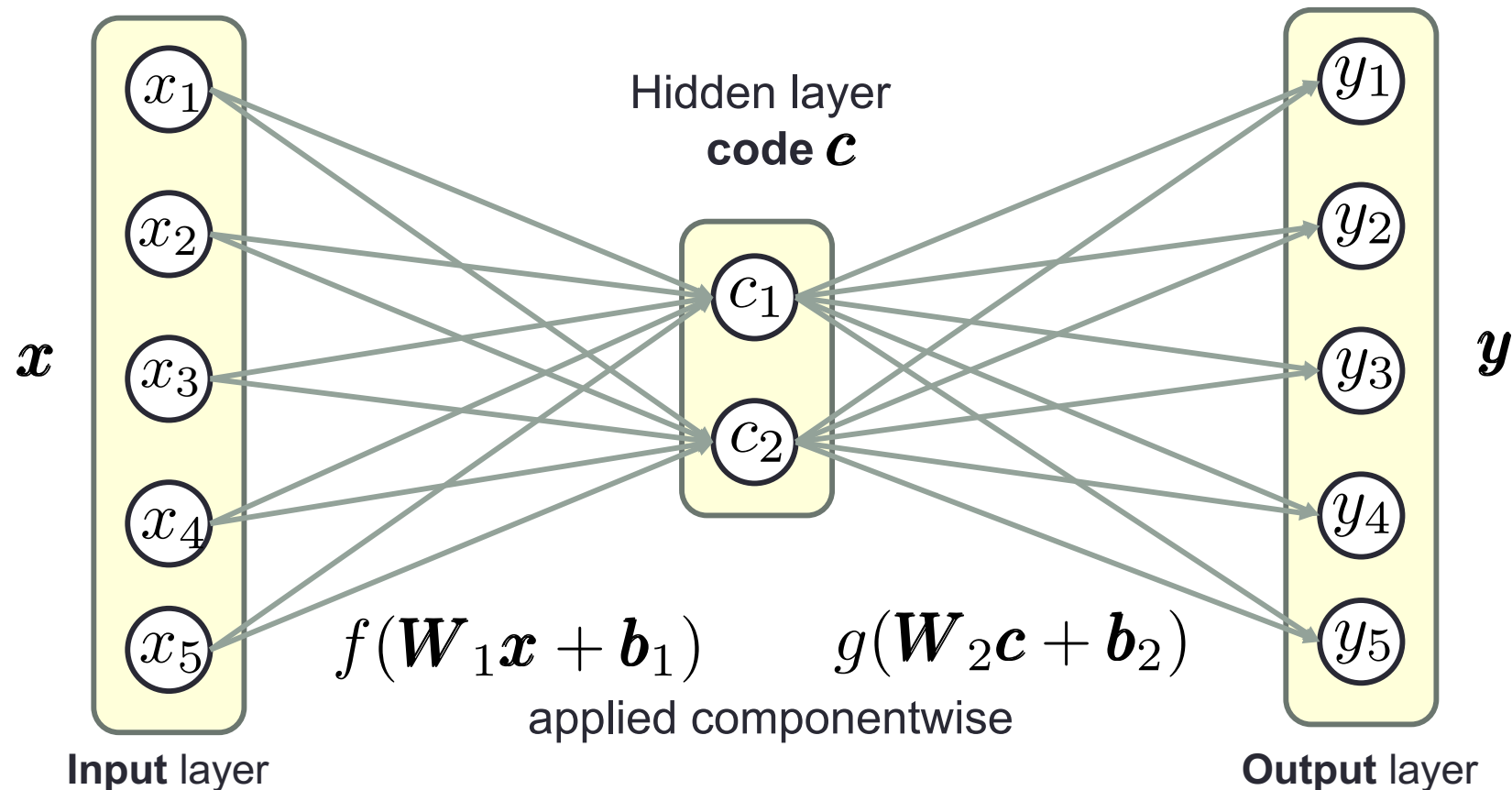
minimum error formulation of PCA

(15)

AUTOENCODERS

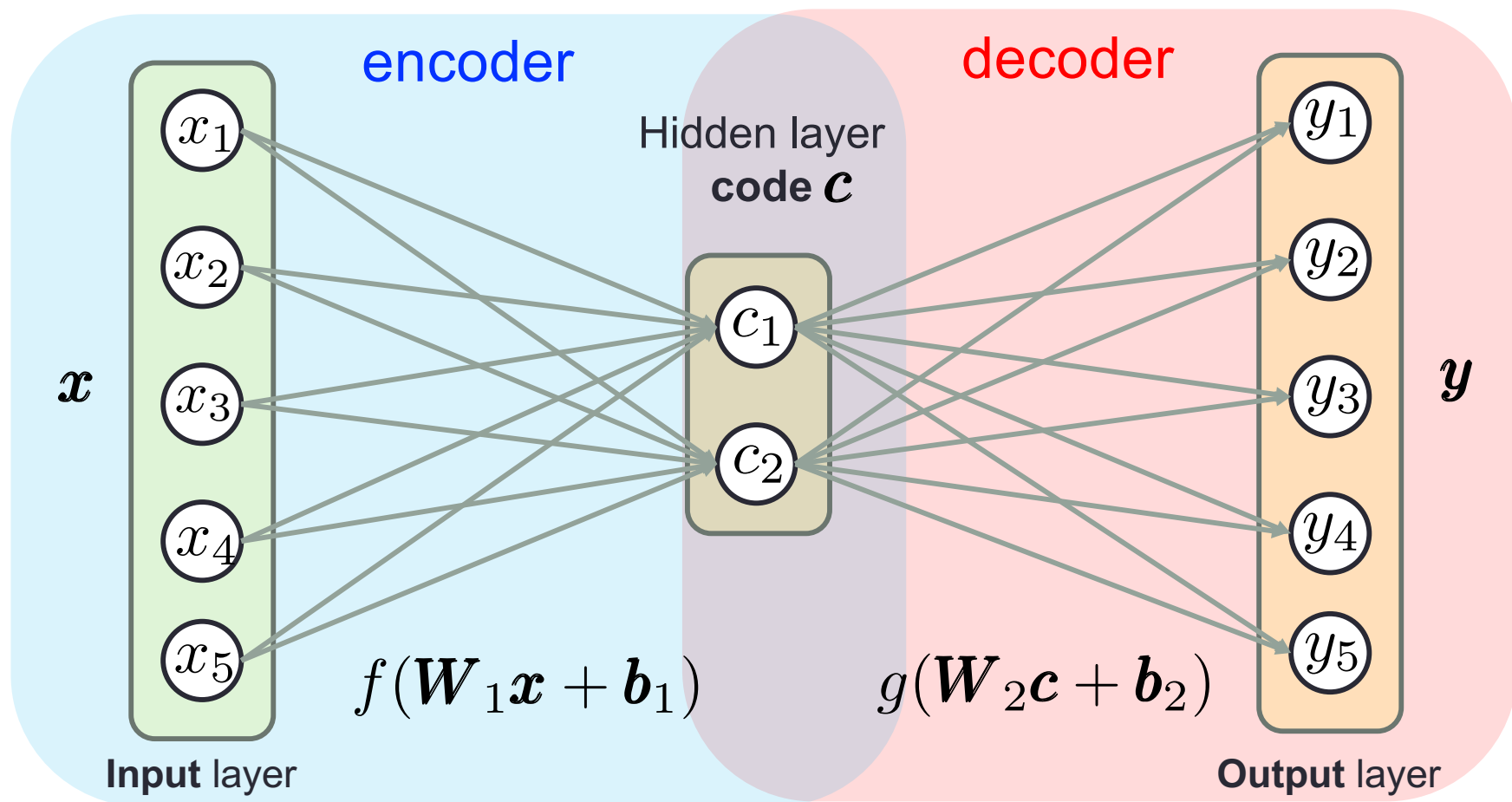
Autoencoder through FFNNs

- Feed Forward Neural Networks (FFNN)
 - Implement functions, **do not have internal states** (memory cells)
 - Artificial neurons organized in layers, signals flow from input (left) to output (right)
 - Structure is **fully connected** (i.e., dense)



Autoencoder through FFNNs

- Encoder-decoder FFNN architecture
 - Intended to reproduce the input



The FFNN autoencoder

- Input vectors $\mathbf{x}_k \in \mathbb{R}^m$, $k = 1, 2, \dots, n$
- Data matrix $\mathbf{X} \in \mathbb{R}^{m \times n}$
- Encoder output
$$\mathbf{c}_k = f(\mathbf{W}_1 \mathbf{x}_k + \mathbf{b}_1) \text{ code vector}$$
 - $f(\cdot)$: componentwise nonlinearity
 - Encoder weights: $\mathbf{W}_1 \in \mathbb{R}^{p \times m}$, $\mathbf{b}_1 \in \mathbb{R}^p$
- Decoder output
$$\mathbf{y}_k = g(\mathbf{W}_2 \mathbf{c}_k + \mathbf{b}_2)$$
 - $g(\cdot)$: componentwise nonlinearity
 - Decoder weights: $\mathbf{W}_2 \in \mathbb{R}^{m \times p}$, $\mathbf{b}_2 \in \mathbb{R}^m$

First setup – the linear autoencoder

- With **f()** and **g()** identities
- Input vectors $\mathbf{x}_k \in \mathbb{R}^m$, $k = 1, 2, \dots, n$
- **Input** data matrix $\mathbf{X} \in \mathbb{R}^{m \times n}$
- **Output** matrix $\mathbf{Y} \in \mathbb{R}^{m \times n}$
- **Code** matrix $\mathbf{C} \in \mathbb{R}^{p \times n}$

$$\mathbf{C} = \mathbf{W}_1 \mathbf{X} + \mathbf{b}_1 \mathbf{1}^T$$

$$\mathbf{Y} = \mathbf{W}_2 \mathbf{C} + \mathbf{b}_2 \mathbf{1}^T \quad (16)$$

with **f()** and **g()** identities (for now)
 $\mathbf{1}$: column vector of all 1s

Autoencoder – objective J

- **Objective:** make each output vector $\mathbf{y}_k$ as close as possible to the corresponding input vector $\mathbf{x}_k$
- **Squared error norm corresponds to (cost function)**

$$J = \sum_{k=1}^n \|\mathbf{x}_k - \mathbf{y}_k\|^2$$

- It can be compactly rewritten using the **F-norm** (see Eq. (3) in this slide set), as follows

$$J = \|\mathbf{X} - \mathbf{Y}\|_F^2 = \|\mathbf{X} - \mathbf{W}_2 \mathbf{C} - \mathbf{b}_2 \mathbf{1}^T\|_F^2 \quad (17)$$

Autoencoder – objective J

- Squared error norm

$$J = \|\mathbf{X} - \mathbf{Y}\|_F^2 = \|\mathbf{X} - \mathbf{W}_2\mathbf{C} - \mathbf{b}_2\mathbf{1}^T\|_F^2$$

- Objective: finding $\mathbf{b}_2$ that minimizes J:

$$\begin{aligned}\mathbf{b}_2^* &= \operatorname{argmin}_{\mathbf{b}_2} \|\mathbf{X} - \mathbf{W}_2\mathbf{C} - \mathbf{b}_2\mathbf{1}^T\|_F^2 \\ &= \operatorname{argmin}_{\mathbf{b}_2} [\operatorname{trace}(\mathbf{A}\mathbf{A}^T)]\end{aligned}$$

- with (using Eq. (4))

$$\begin{cases} \|\mathbf{A}\|_F^2 = \operatorname{trace}(\mathbf{A}\mathbf{A}^T) \\ \mathbf{A} \triangleq \mathbf{X} - \mathbf{W}_2\mathbf{C} - \mathbf{b}_2\mathbf{1}^T \end{cases}$$

Autoencoder – optimal bias $\mathbf{b}_2$

$$\mathbf{b}_2^* = \operatorname{argmin}_{\mathbf{b}_2} \|\mathbf{X} - \mathbf{W}_2 \mathbf{C} - \mathbf{b}_2 \mathbf{1}^T\|_F^2$$

- Since

$$\|\mathbf{A}\|_F^2 = \operatorname{trace}(\mathbf{A} \mathbf{A}^T)$$

$$\nabla_{\mathbf{b}_2} (\operatorname{trace}(\mathbf{A} \mathbf{A}^T)) = \mathbf{0}$$

- Leads to (see Proof. in [Appendix 3](#))

$$\mathbf{b}_2^* = \frac{1}{n} (\mathbf{X} - \mathbf{W}_2 \mathbf{C}) \mathbf{1} \quad (18)$$

Autoencoder – objective J

- Replacing optimal $\mathbf{b}_2^*$ (Eq. (18)), we obtain

$$\begin{aligned} J &= \|\mathbf{X} - \mathbf{Y}\|_F^2 = \|\mathbf{X} - \mathbf{W}_2 \mathbf{C} - \mathbf{b}_2^* \mathbf{1}^T\|_F^2 = \\ &= \|\mathbf{X}' - \mathbf{W}_2 \mathbf{C}'\|_F^2 \end{aligned} \quad (19)$$

- with

$$\mathbf{X}' = \mathbf{X} \left(\mathbf{I} - \frac{\mathbf{1}\mathbf{1}^T}{n} \right) \quad (20)$$

$$\mathbf{C}' = \mathbf{C} \left(\mathbf{I} - \frac{\mathbf{1}\mathbf{1}^T}{n} \right) \quad (21)$$

these are
zero mean
matrices
(see next slide)

Autoencoder – effect of $\mathbf{b}_2^*$

- **Note:** average vectors for input, hidden and output units are

$$\bar{\mathbf{x}} = \frac{\mathbf{X}\mathbf{1}}{n} \quad \bar{\mathbf{c}} = \frac{\mathbf{C}\mathbf{1}}{n} \quad \bar{\mathbf{y}} = \frac{\mathbf{Y}\mathbf{1}}{n}$$

- From these, it descends

$$\mathbf{X}' = \mathbf{X} \left(\mathbf{I} - \frac{\mathbf{1}\mathbf{1}^T}{n} \right) = \mathbf{X} - \bar{\mathbf{x}}\mathbf{1}^T \quad (22)$$

$$\mathbf{C}' = \mathbf{C} \left(\mathbf{I} - \frac{\mathbf{1}\mathbf{1}^T}{n} \right) = \mathbf{C} - \bar{\mathbf{c}}\mathbf{1}^T \quad (23)$$

- The optimal bias vector $\mathbf{b}_2$ reduces the training problem (17) **to zero-average patterns**

Autoencoder – effect of $\mathbf{b}_2^*$

- **Moreover:** recalling (17) and (18)

$$\mathbf{Y} = \mathbf{W}_2 \mathbf{C} + \mathbf{b}_2^* \mathbf{1}^T \quad \mathbf{b}_2^* = \frac{1}{n} (\mathbf{X} - \mathbf{W}_2 \mathbf{C}) \mathbf{1} \quad \bar{\mathbf{x}} = \frac{\mathbf{X} \mathbf{1}}{n}$$

$$\Rightarrow \mathbf{Y} = \mathbf{W}_2 \mathbf{C} + \left(\bar{\mathbf{x}} - \frac{\mathbf{W}_2 \mathbf{C} \mathbf{1}}{n} \right) \mathbf{1}^T \quad (24)$$

$$\mathbf{Y} \mathbf{1} = \mathbf{W}_2 \mathbf{C} \mathbf{1} + \left(\bar{\mathbf{x}} - \frac{\mathbf{W}_2 \mathbf{C} \mathbf{1}}{n} \right) \mathbf{1}^T \mathbf{1}$$

$(\mathbf{1}^T \mathbf{1} = n)$

$$\mathbf{Y} \mathbf{1} = n \bar{\mathbf{x}}$$

Autoencoder – effect of $\mathbf{b}_2^*$

- **Moreover:** recalling (17) and (18)

$$\mathbf{Y} = \mathbf{W}_2 \mathbf{C} + \mathbf{b}_2^* \mathbf{1}^T \quad \mathbf{b}_2^* = \frac{1}{n} (\mathbf{X} - \mathbf{W}_2 \mathbf{C}) \mathbf{1} \quad \bar{\mathbf{x}} = \frac{\mathbf{X} \mathbf{1}}{n}$$

$$\Rightarrow \mathbf{Y} = \mathbf{W}_2 \mathbf{C} + \left(\bar{\mathbf{x}} - \frac{\mathbf{W}_2 \mathbf{C} \mathbf{1}}{n} \right) \mathbf{1}^T \quad (24)$$

$$\mathbf{Y} \mathbf{1} = \mathbf{W}_2 \mathbf{C} \mathbf{1} + \left(\bar{\mathbf{x}} - \frac{\mathbf{W}_2 \mathbf{C} \mathbf{1}}{n} \right) \mathbf{1}^T \mathbf{1}$$

$$\bar{\mathbf{y}} = \frac{\mathbf{Y} \mathbf{1}}{n} = \bar{\mathbf{x}} \quad (25)$$

Optimal bias scales input and output vectors to the **same average value**

Minimizing J

- So far, we have obtained

$$J = \|\mathbf{X}' - \mathbf{W}_2 \mathbf{C}'\|_F^2 \quad \Rightarrow \quad J_{\min} = \min_{\mathbf{W}_2 \mathbf{C}'} \|\mathbf{X}' - \mathbf{W}_2 \mathbf{C}'\|_F^2$$

?

- Let the SVD of $\mathbf{X}'$ be

$$\mathbf{X}' = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$$

- Assume that $\mathbf{W}_2$ has **low rank** $p < m$ (usually verified)
- Then, from the **low-rank approximation theorem**, it descends that **the best p-rank approximation** is

$$\mathbf{W}_2 \mathbf{C}' = \mathbf{U} \mathbf{\Sigma}_p \mathbf{V}^T \quad (26)$$

optimal approx.

Let's look at the first linear layer

- For the first layer, it holds $\mathbf{C} = \mathbf{W}_1 \mathbf{X} + \mathbf{b}_1 \mathbf{1}^T$
- Multiplying both sides by $\left(\mathbf{I} - \frac{\mathbf{1}\mathbf{1}^T}{n} \right)$
- Using (22) and (23), leads to (as $\mathbf{1}^T \mathbf{1} = n$)

$$\mathbf{C}' = \mathbf{W}_1 \mathbf{X}' + \mathbf{b}_1 \mathbf{1}^T \left(\mathbf{I} - \frac{\mathbf{1}\mathbf{1}^T}{n} \right) = \mathbf{W}_1 \mathbf{X}' \quad (27)$$

- which does not depend on $\mathbf{b}_1$ and therefore shows that $\mathbf{b}_1$ is arbitrary

Finding the optimal matrices

- Hence, the error function for the linear AE becomes

$$J = \|\mathbf{X}' - \mathbf{W}_2 \mathbf{C}'\|_F^2 = \|\mathbf{X}' - \mathbf{W}_2 \mathbf{W}_1 \mathbf{X}'\|_F^2 \quad (28)$$

- J is minimized with (from (26) and (27))

$$\mathbf{W}_2 \mathbf{C}' = \mathbf{W}_2 \mathbf{W}_1 \mathbf{X}' = \mathbf{U} \Sigma_p \mathbf{V}^T \quad (29)$$

- The following is a solution (i.e., minimizes J)

$$\begin{cases} \mathbf{W}_2 = \mathbf{U} \mathbf{T}^{-1} \\ \mathbf{C}' = \mathbf{W}_1 \mathbf{X}' = \mathbf{T} \Sigma_p \mathbf{V}^T \end{cases} \quad (30)$$

- For an arbitrary orthogonal $p \times p$ matrix $\mathbf{T}$
- Hence, the solution to is not unique

Putting it all together

- The error function for the **linear AE** becomes

$$J = \|\mathbf{X}' - \mathbf{W}_2 \mathbf{C}'\|_F^2 = \|\mathbf{X}' - \boxed{\mathbf{W}_2} \boxed{\mathbf{W}_1} \mathbf{X}'\|_F^2$$

going from latent space back to original space moving input to latent space

- multiple solutions exist for the two matrices, **back propagation** finds one
- With **PCA** we have

$$J = \|\mathbf{X}' - \mathbf{W} \mathbf{W}^T \mathbf{X}'\|_F^2$$

- Optimal **W** is uniquely determined by SVD applied to **Cov(X')**
- The columns of **W** are the principal vectors (eigenvalues of **Cov(X')**), ordered according to the amplitude of the eigenvalues of **Cov(X')**

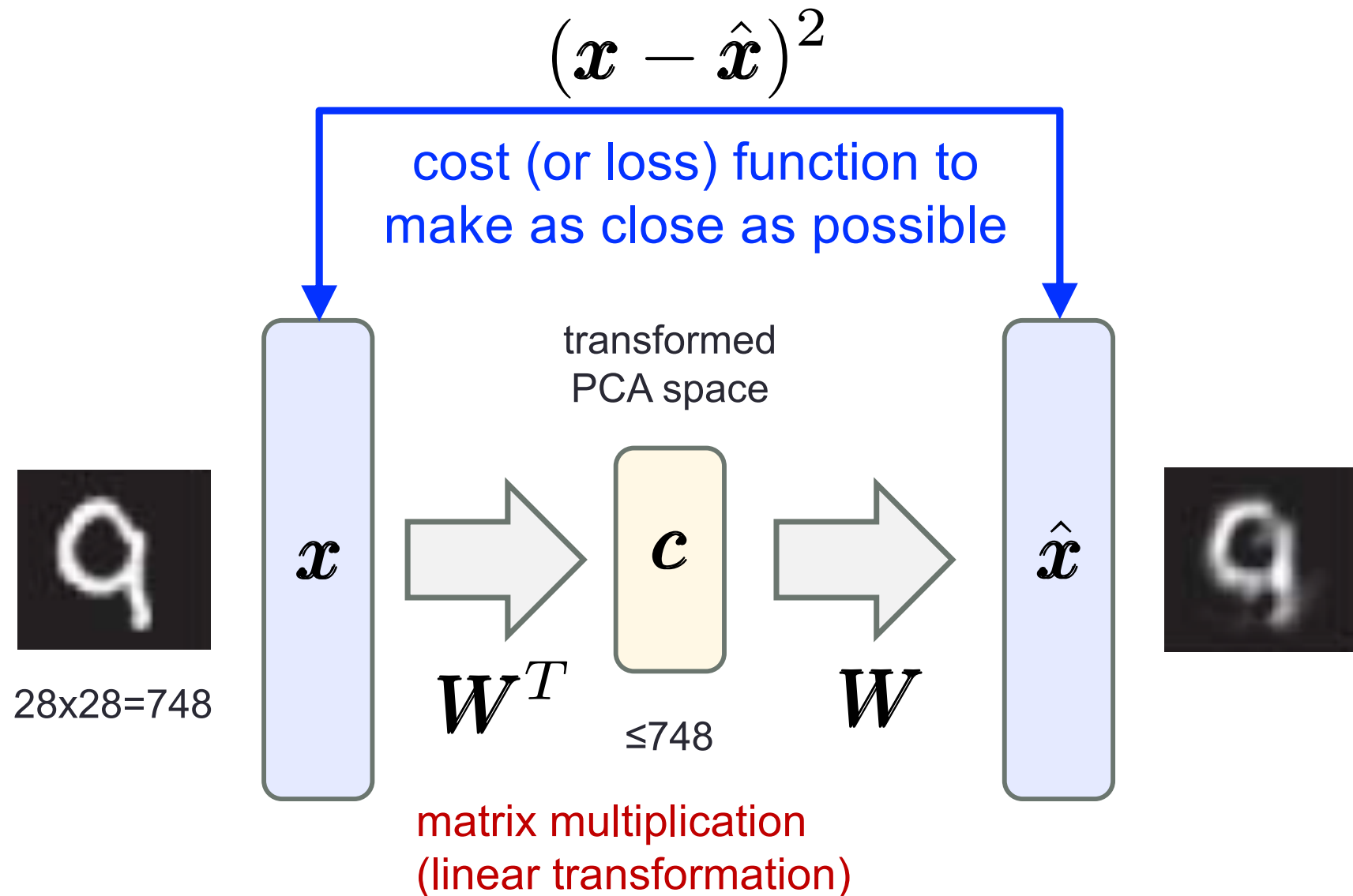
Take home message

- The linear **AE behaves as a PCA**, in the sense that they minimize the same error between original and reconstructed data vectors
- They both behave as **data compressors** – the compressed representation is the inner vector **c** for the AE (spanning the same subspace)
- However, the AE
 - unlike PCA, the coordinates of **c** can be **correlated and not necessarily sorted** in descending order of variance (eigenvalues)
 - **does not ensure** that entries of vector **c** are **uncorrelated**
 - matrix **W₂** is, in general, equal to

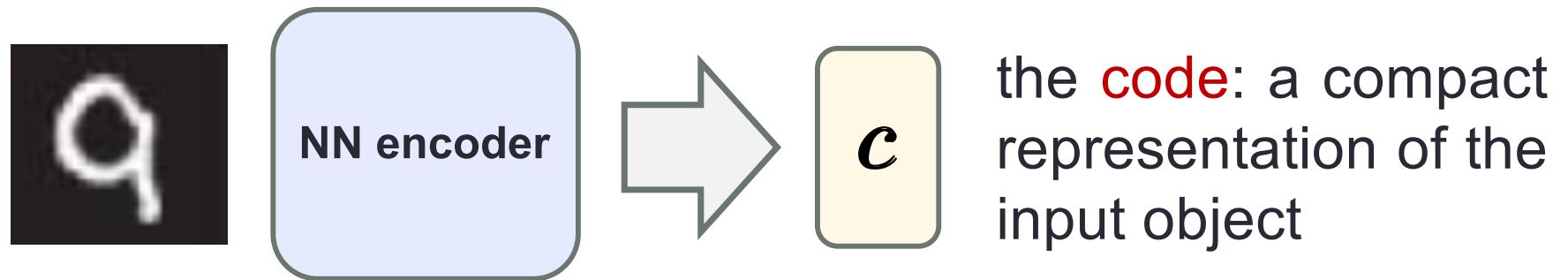
$$\mathbf{W}_2 = \mathbf{W}\mathbf{O}$$

- where **W** is the matrix found by PCA, **O** is an orthogonal matrix

Principal Component Analysis (PCA)

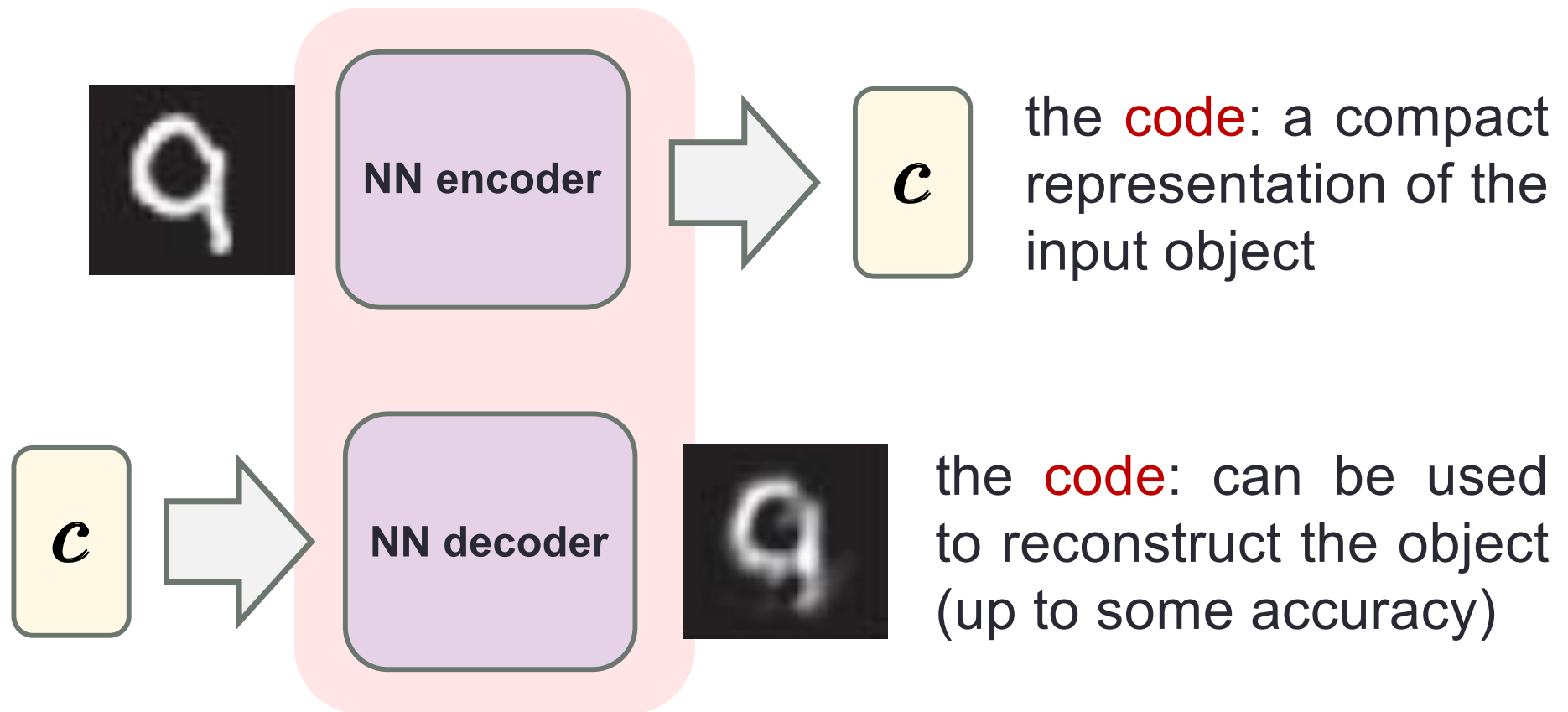


Autoencoder [Hinton06]



[Hinton06] G. E. Hinton, R. R. Salakhutdinov, "Reducing the Dimensionality of Data with Neural Networks," Science, Vol. 313, July 2006.

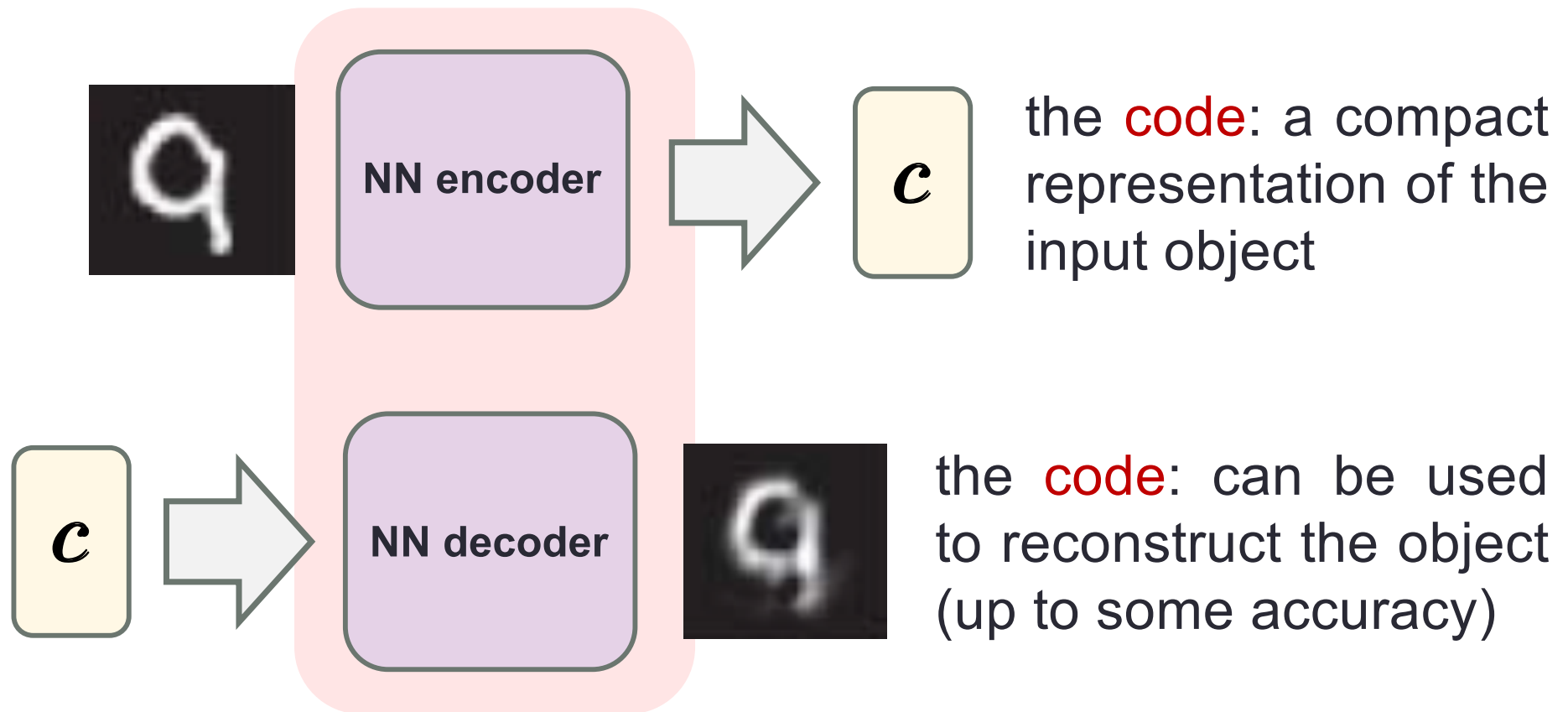
Autoencoder



Key point 1: encoder and decoder

- are **jointly trained**

Autoencoder



Key point 2: encoder and decoder

- are **non linear** (usually via **sigmoid**, **tanh**, **ReLU**, activations)

Autoencoder caveats



- If after training $x = y$ everywhere
 - This is useless and must be avoided !!!
- Autoencoders (AE) should be trained such that
 - They are unable to learn to copy perfectly
 - They only copy **approximately**
 - And **only copy** input that resembles **valid** input data
- In this way AE
 - Must **prioritize** *which aspects* of the input should be copied
 - Usually learn useful properties of the data

Example: linear multiplication by identity matrix copies perfectly every input vector but **is useless**

AE training

- Unsupervised

- Label (target output) is the input data itself
- Are trained using **gradient descent** as any FFNN

- Standard gradient descent techniques

- **Batch-mode gradient descent**: all data points considered to compute the true error derivative wrt the FFNN weights
- **Stochastic gradient descent (SGD)**: 1) one input vector at a time is fed to the FFNN, 2) gradient computed solely based on it, 3) network weights are updated using gradient descent of this pointwise gradient, 4) reiterate for all points
- **Mini-batches**: between batch-mode and SGD

Learning strategies

- Objective

- Prevent the AE from *just copying* the data

- Solutions

- Limit the AE approximation *capacity*

- Popular strategies

- Undercomplete AEs
 - Sparse AEs
 - Denoising AEs

Undercomplete AE

- Input x
- Output $y = g(f(x))$
- Loss (error) $\mathcal{L}(x, g(f(x)))$
- Under completeness
 - Code dimension $p \ll m$ input dimension
 - Forces the AE to capture the most salient data features
- Special case
 - Linear encoder/decoder and quadratic loss: the AE learns to span the same subspace as PCA (they are equivalent)

Sparse AE

- Input x
- Output $y = g(f(x))$
- Loss (error) $\mathcal{L}(x, g(f(x)))$
- Training criterion $\mathcal{L}(x, g(f(x))) + \Omega(c)$

Sparsity penalty:

- It is a **regularizer** term
- Expresses a **preference over functions**
- For instance (Laplacian prior), we have:

$$\Omega(c) = \lambda \sum_i |c_i|$$

Laplacian prior (1/2)

- Idea

- See the sparse AE framework as approximating ML training of a *generative model* that has latent variables (code $\mathbf{c}$)
- **Explicit joint distribution** (*input $\mathbf{x}$ and latent variable $\mathbf{c}$*)

$$p_{\text{model}}(\mathbf{x}, \mathbf{c}) = p_{\text{model}}(\mathbf{c})p_{\text{model}}(\mathbf{x}|\mathbf{c})$$

$$\log p_{\text{model}}(\mathbf{x}, \mathbf{c}) = \log p_{\text{model}}(\mathbf{c}) + \log p_{\text{model}}(\mathbf{x}|\mathbf{c})$$

- We impose a Laplacian prior over c_i (assuming c_i i.i.d.)

$$p_{\text{model}}(c_i) = \frac{\lambda}{2} e^{-\lambda|c_i|}$$

Laplacian prior (2/2)

- Laplacian prior over c_i

i.i.d. assumption

$$p_{\text{model}}(c_i) = \frac{\lambda}{2} e^{-\lambda|c_i|} \quad p_{\text{model}}(\mathbf{c}) = \prod_i p_{\text{model}}(c_i)$$

- As a training cost, we take $-\log$ of the prior

$$-\log p_{\text{model}}(\mathbf{c}) = \sum_i \left(\lambda|c_i| - \log \frac{\lambda}{2} \right) = \Omega(\mathbf{c}) + \text{constant}$$

- Minimizing the cost:** amounts to maximizing the prior pdf
- Other priors are possible:** lead to different penalties
- This show why the features learned by an AE are useful:** they describe the latent variables that explain the input

Denoising AE (1/3)

- Rather than constrain the representation (the code)
 - Train the AE for a more challenging task:
 - cleaning partially corrupted input (denoising)
- From [Vincent10]:

“a good representation is one that can be obtained robustly from a corrupted input and that will be useful for recovering the corresponding clean input...”

[Vincent10] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, P.A. Manzagol, “Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion,” *Journal of Machine Learning Research*, 2010.

Denoising AE (1/3)

- Rather than constrain the representation (the code)
 - Train the AE for a more challenging task:
 - cleaning partially corrupted input (denoising)
- From [Vincent10]:

“...our goal is not the task of denoising per se. Rather, denoising is advocated and investigated as a training criterion for learning to extract useful features that will constitute better higher level representations...”

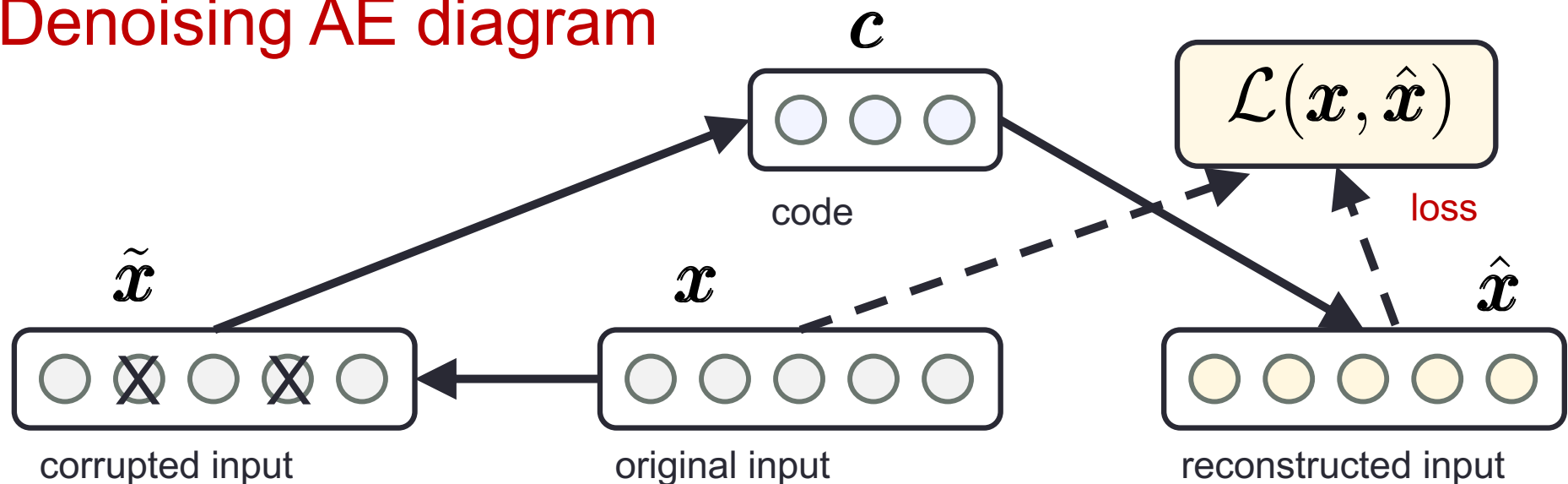
[Vincent10] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, P.A. Manzagol, “Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion,” *Journal of Machine Learning Research*, 2010.

Denoising AE (2/3)

- Rationale

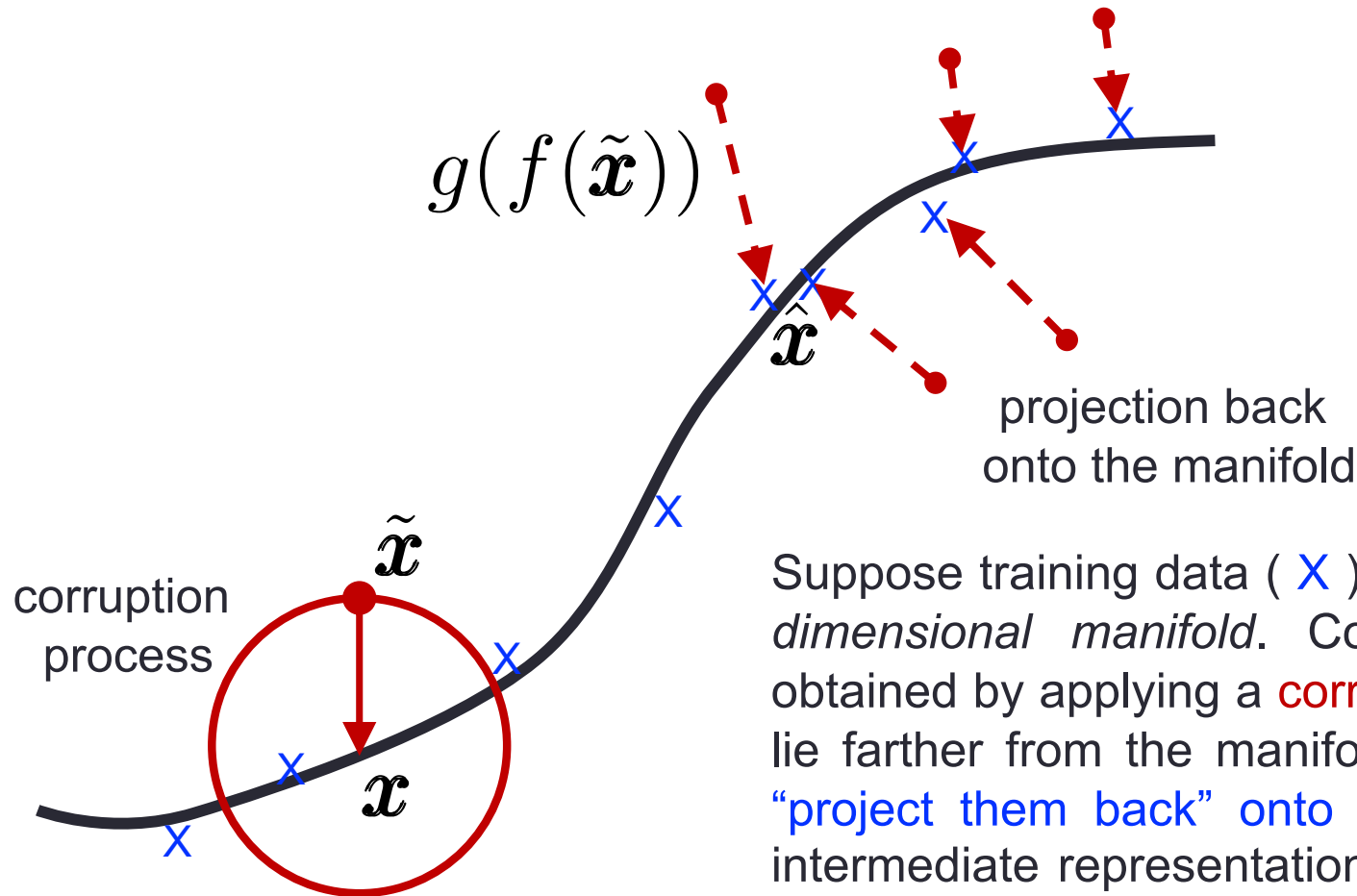
- 1) It is expected that a high-level representation should be rather stable under a corruption of the input
- 2) It is expected that performing the denoising task requires extracting features that capture useful features in the input distribution

- Denoising AE diagram



Denoising AE (3/3)

Geometrical interpretation - manifold learning



Suppose training data (X) concentrate near a *low-dimensional manifold*. Corrupted examples (•) obtained by applying a **corruption process** generally lie farther from the manifold. The model learns to “**project them back**” onto the manifold. Thus, the intermediate representation $\mathbf{c} = f(\mathbf{x})$ may be seen as a coordinate system for points on the manifold

AE for Missing Data Imputation

Algorithms' Family	Algorithm & Article	Score
Matrix Completion [△]	Singular Value Thresholding (Tran et al., 2017)	4
	SoftImpute (Tran et al., 2017)	4
	OptSpace (Tran et al., 2017)	4
Evolutionary ^{△□}	Genetic Algorithms (GA) (Tran et al., 2017)	4
	Genetic Algorithms (Malek et al., 2018)	4
Connectionist [△]	Denoising Autoencoder (Tran et al., 2017)	4
	Stacked Denoising Autoencoder (Tran et al., 2017)	4
	Multi-modal Autoencoder (Tran et al., 2017)	4
	Deep Canonically Correlated Autoencoders (Tran et al., 2017)	3
	Variational Autoencoder (Ma et al., 2019)	4
Other [□]	Orthogonal Matching Pursuit (Malek et al., 2018)	4
	Basis Pursuit (Malek et al., 2018)	4

Imputation in images

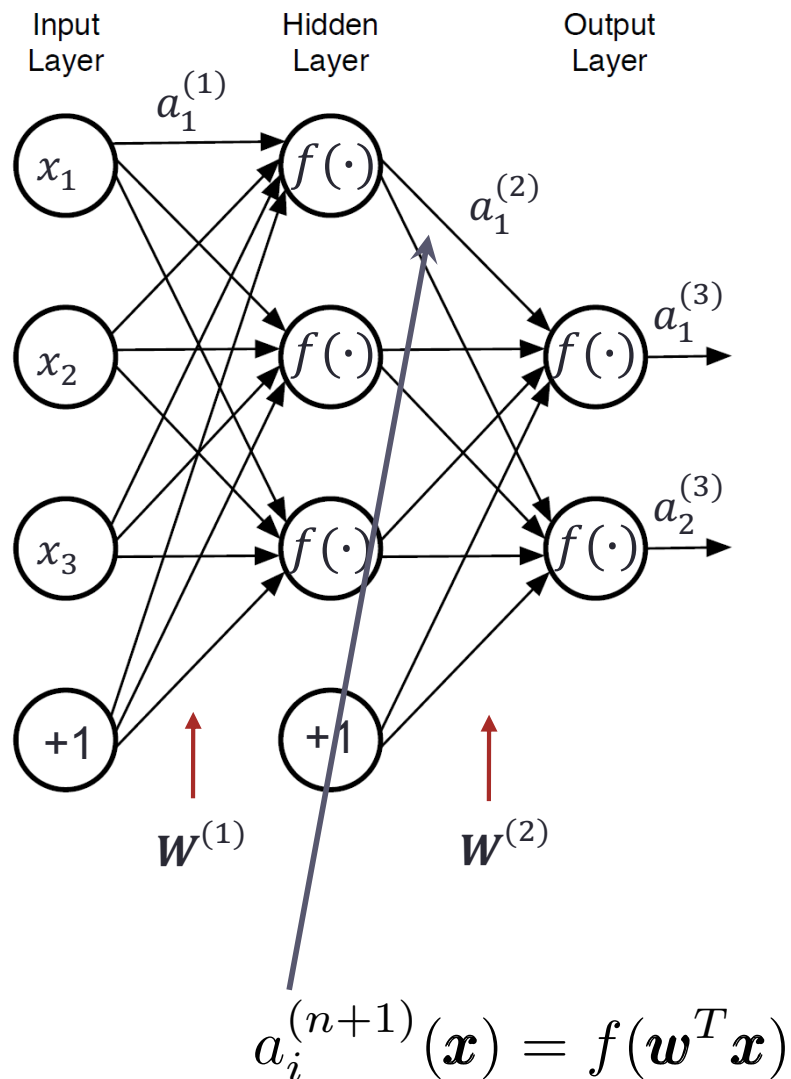
1. AE has worse results
2. AE has same results
3. AE has marginally better results
4. AE has significantly better results

[Pereira2020] R. C. Pereira, M. S. Santos, P. P. Rodriguez, P. H. Abreu, "Reviewing Autoencoders for Missing Data Imputation: Technical Trends, Applications and Outcomes," *Journal of Artificial Intelligence Research*, Vol. 69, 2020.

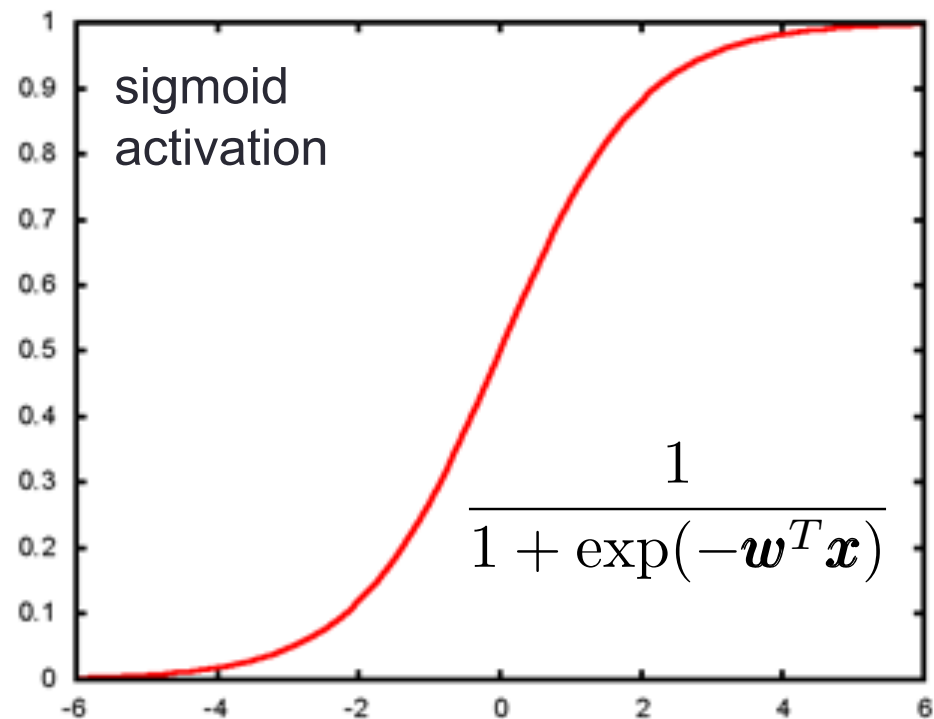
VECTOR QUANTIZATION AS AUTOENCODER-BASED COMPRESSION – THE CASE OF ECG SIGNALS

[DelTesta2015] Davide Del Testa, Michele Rossi, “Lightweight Lossy Compression of Biometric Patterns via Denoising Autoencoders,” *IEEE Signal Processing Letters*, 2015.

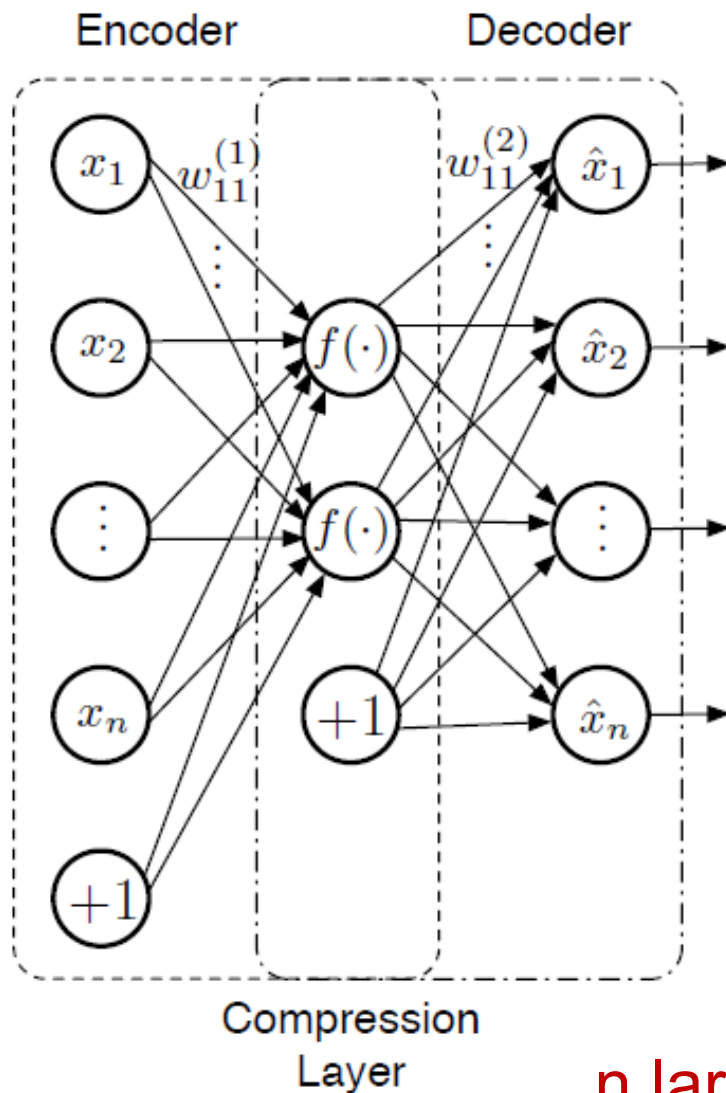
Neural networks



- Feed Forward NN
- Layers of neurons
- Non-linear activation functions
- Set of **weights**



Autoencoders



- Unsupervised learning
- Same no. of input & output neurons
- Target values = input
- **Goal:** learn to reconstruct the input

$$\mathbf{x} \in \mathbb{R}^n$$

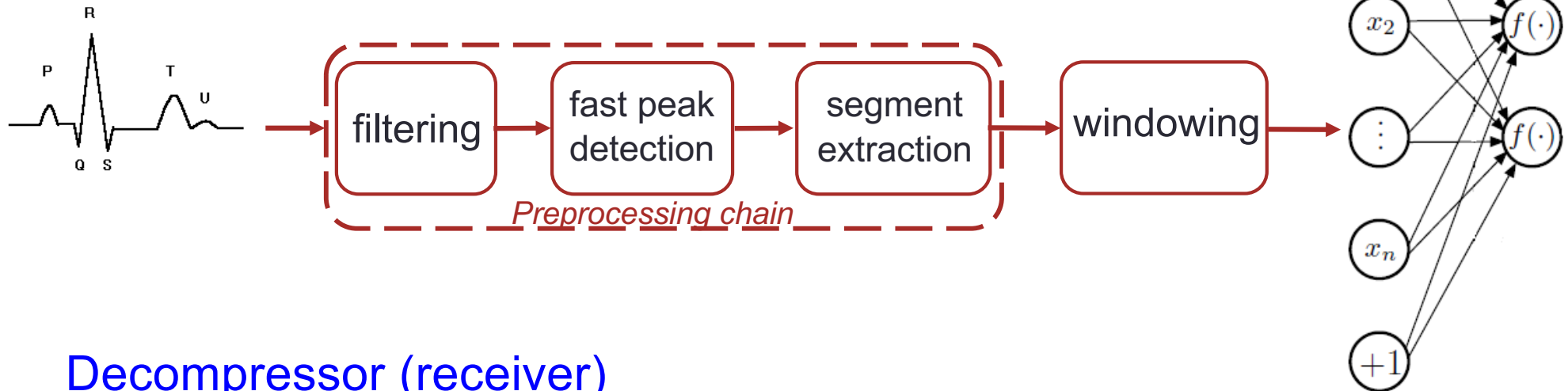
mapped onto $\mathbf{x}' \in \mathbb{R}^c, c < n$

➡ c hidden units

n larger than 250 ECG samples / segment

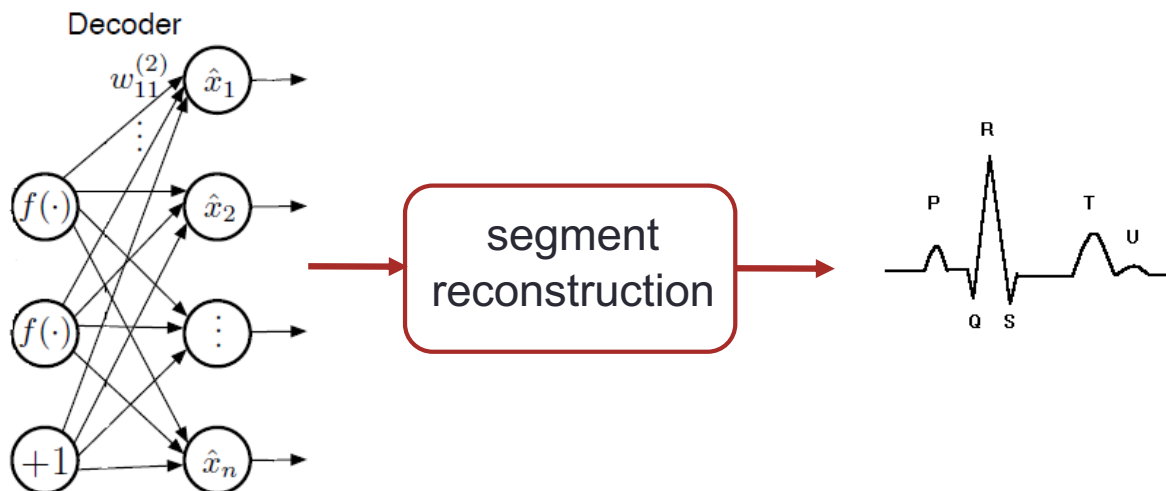
Compression architecture

Compressor (transmitter)



c values TX

Decompressor (receiver)



weight set W
computed offline
(training examples)

Performance metrics (1/2)

E1 - Energy associated with compression

- We count the number of operations (divisions, additions, comparisons)
- Translate them into the corresponding number of clock cycles
- From clock cycles → energy consumption
- **MCU:** ARM Cortex M4

E2 - Energy associated with transmission / reception

- Consider the compressed data stream
- Compute the energy consumption associated with TX / RX
- **Radio:** Texas Instruments CC2541 (Bluetooth SoC)



total energy $E1+E2$

Performance metrics (2/2)

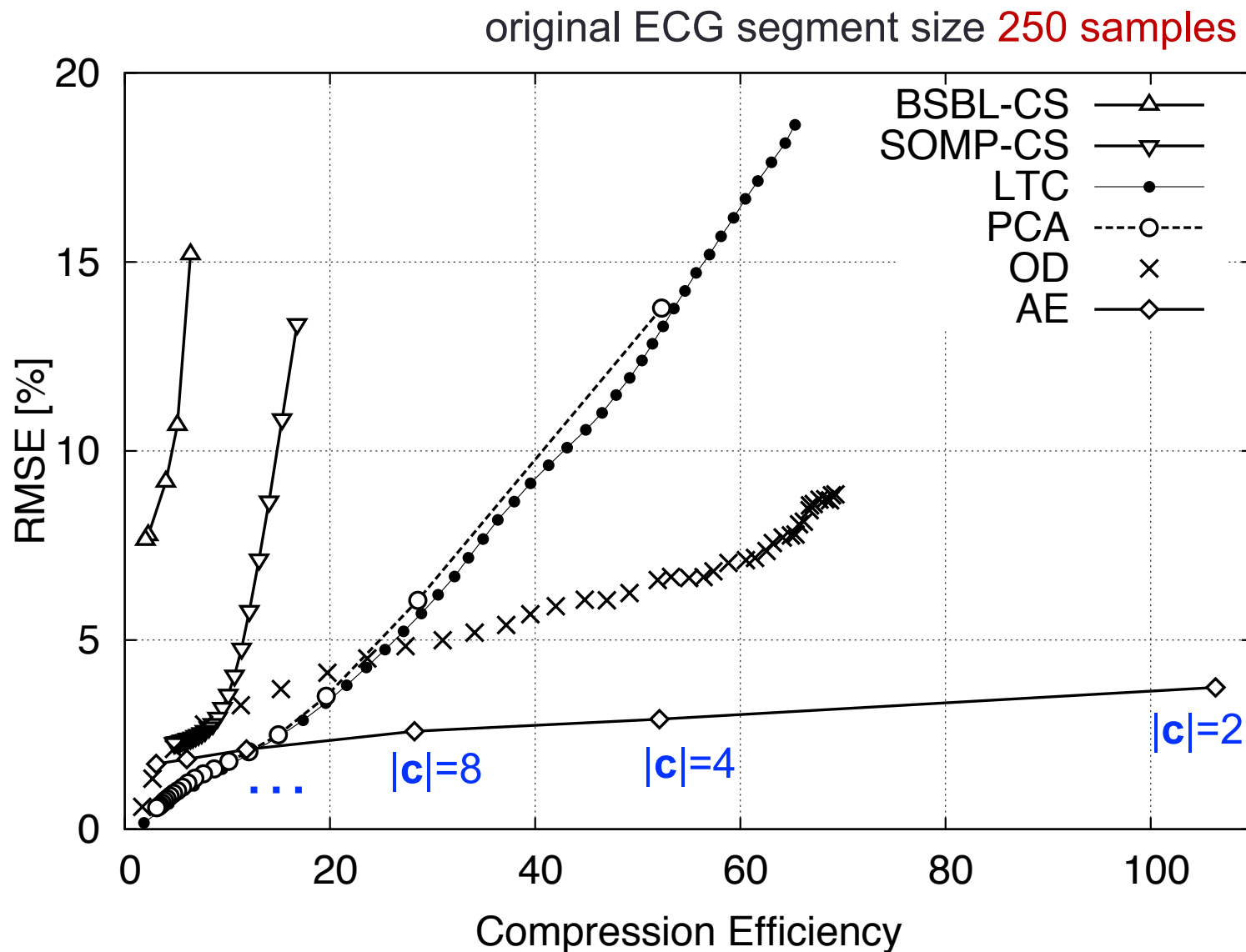
Representation accuracy

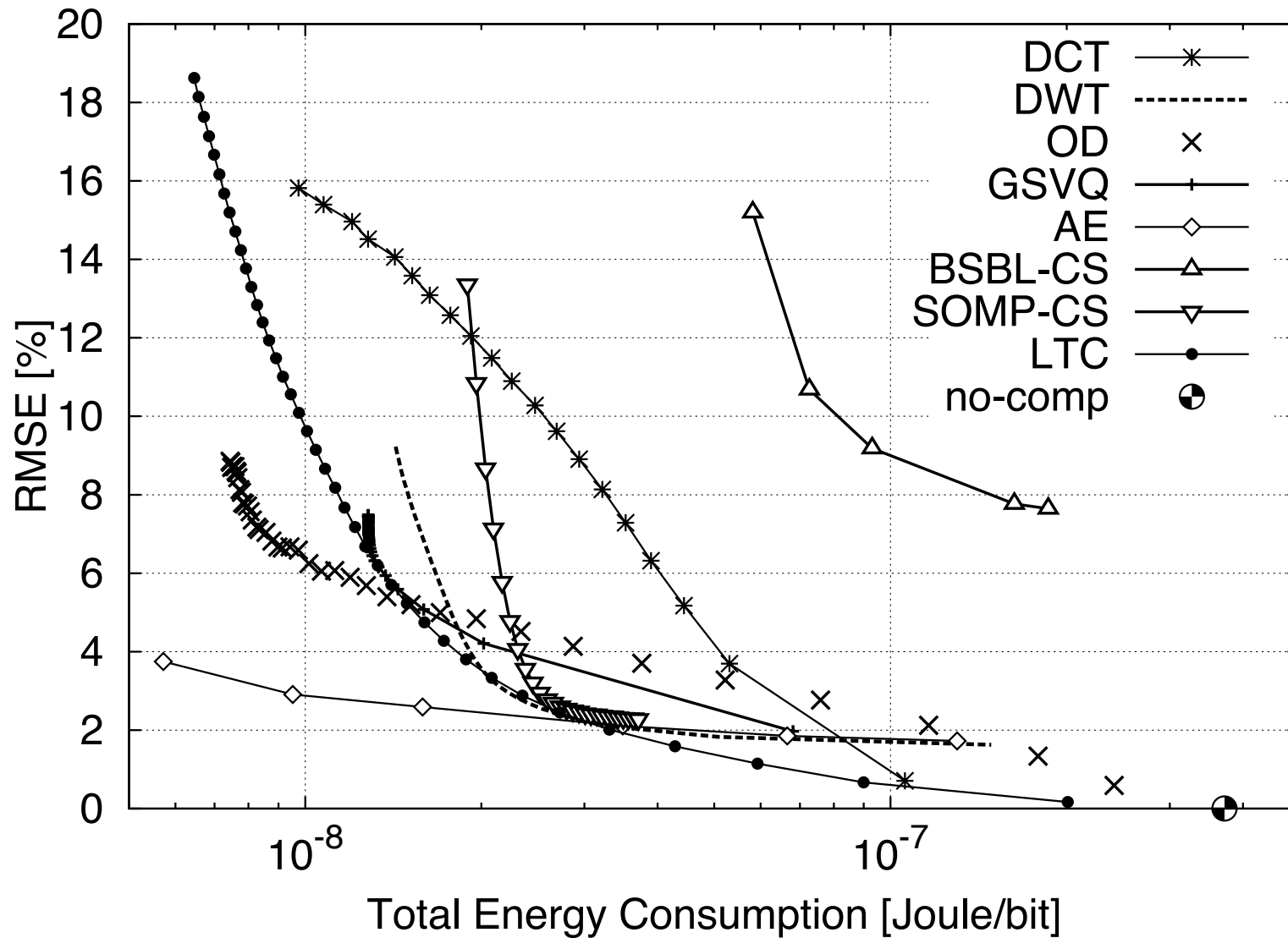
- Root Mean Square Error (RMSE)
- Expressed as % of the p2p signal's amplitude

Compression efficiency

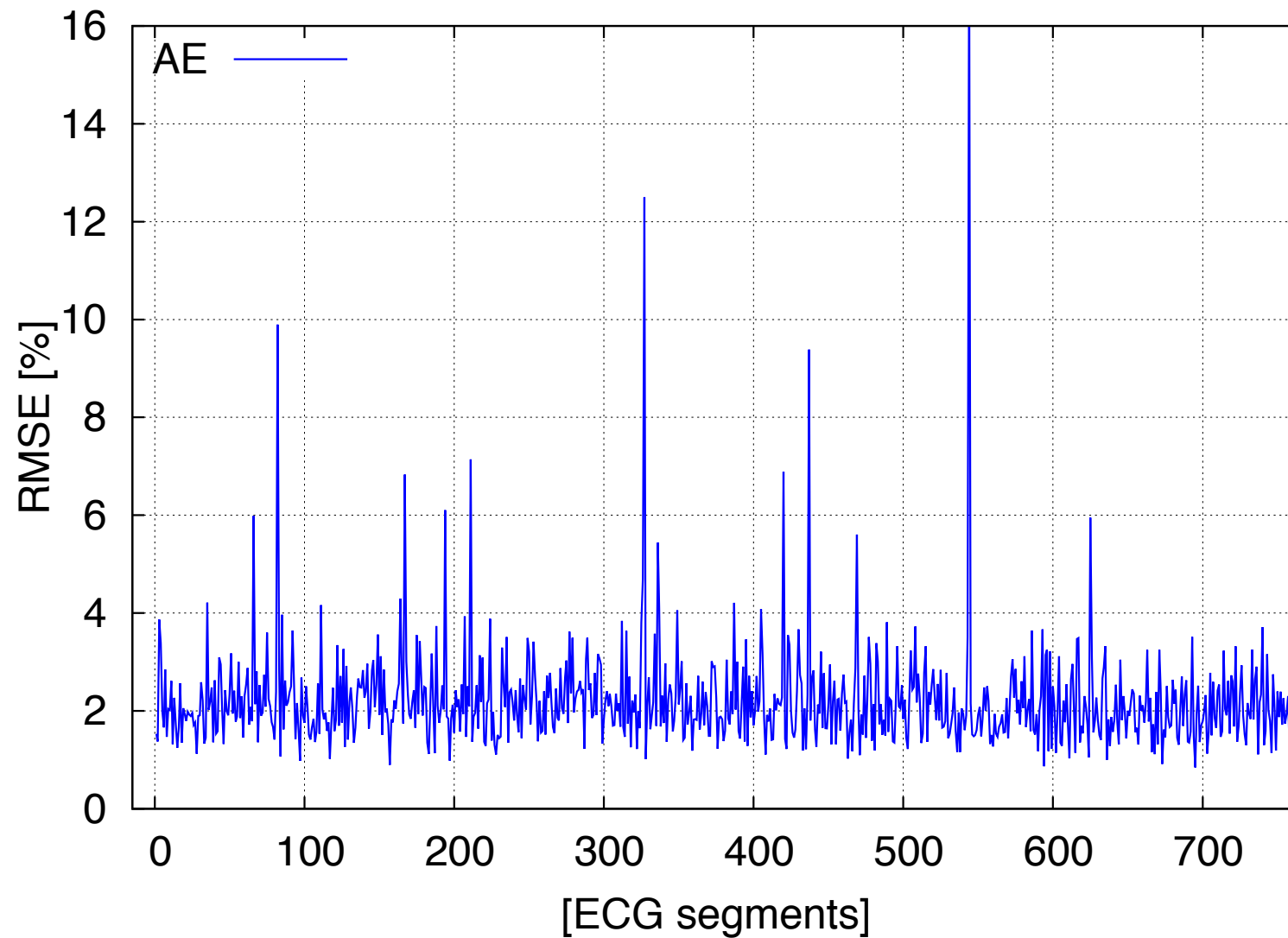
$$CE = \frac{\text{\#bits in the original stream}}{\text{\#bits in the compressed stream}}$$

Denoising AE – example results (ECG)





Where AE fails



Bibliography

[Amhed1975] N. Ahmed, K. R. Rao, “Orthogonal transforms for digital signal processing,” Springer, New York Berlin Heidelberg, 1975.

[Baldiand1989] P. Baldiand, K. Hornik, “Neural networks and principal component analysis: Learning from examples without local minima,” *Neural Networks*, vol. 2(1), pp. 53–58, 1989.

[Hinton06] G. E. Hinton and R. R. Salakhutdinov, “Reducing the Dimensionality of Data with Neural Networks,” *Science*, July 2006.

[Vincent04] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, P.A. Manzagol, “Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion,” *Journal of Machine Learning Research*, 2010.

[DelTesta15] D. Del Testa, M. Rossi, “Lightweight Lossy Compression of Biometric Patterns via Denoising Autoencoders,” *IEEE Signal Processing Letters*, Vol. 22, No. 12, September 2015.

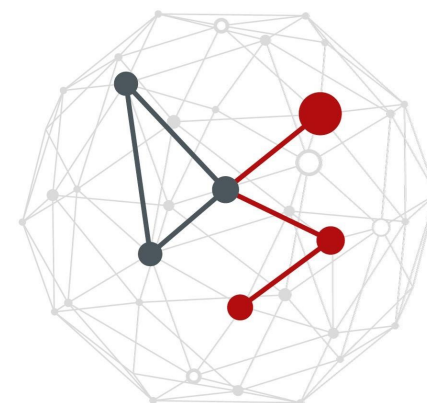
[Plaut2018] E. Plaut, “From Principal Subspaces to Principal Components with Linear Autoencoders,” Technical Report, arXiv:1804.10253 [stat.ML], [v3] December 2018.

AUTOENCODERS

Michele Rossi

michele.rossi@dei.unipd.it

Dept. of Information Engineering
University of Padova, IT



APPENDIX 1 – WHY SQUARE ERRORS?

Square vs absolute errors

- Absolute is often “what we care about”
 - Example: you buy a stock of items to sell them into the future and gain some money from it. If P_{paid} is the price paid to buy the stock and P_{pred} is the future predicted cost, the **money you gain is**

$$P_{\text{pred}} - P_{\text{paid}}$$

(absolute) and **not the square difference**

- The same holds for many other practical examples...
- However, the square error **has many appealing properties** which make its use convenient mathematically
- Let's see some of them...

What does not hold for the absolute error

- If X is a r.v.

- The estimator of X that minimizes the square norm is the mean $E[X]$, whereas the estimator that minimizes the absolute difference is the median $m(X)$. The mean has much nicer properties than the median, e.g., if Y is also a r.v., it holds $E[X+Y] = E[X] + E[Y]$

- If $\mathbf{x}=(x_1, x_2)$ is a vector

- If you have a vector $\mathbf{x}=(x_1, x_2)^T$ estimated by $\mathbf{w}=(w_1, w_2)^T$. For the squared error it does not matter whether you consider the components separately or together, i.e.,

$$\|\mathbf{x} - \mathbf{w}\|^2 = (x_1 - w_1)^2 + (x_2 - w_2)^2$$

- You cannot do this with the absolute error. Moreover, this means that the squared error is independent of reparameterization, i.e.,
- If we define a new vector $\mathbf{y}=(x_1+x_2, x_1-x_2)^T$ the minimum squared deviance estimators for $\mathbf{x}$ and $\mathbf{y}$ are the same

What does not hold for the absolute error

- Independent r.v.s. X and Y

- Variances (expected squared errors) add, i.e.,

$$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$$

- Differentiability

- The absolute error is $\text{MAE} \triangleq |x_{\text{pred}} - x_{\text{true}}|$

$$\frac{d\text{MAE}}{dx_{\text{pred}}} = \begin{cases} +1 & x_{\text{pred}} > x_{\text{true}} \\ -1 & x_{\text{pred}} < x_{\text{true}} \\ \text{undefined} & x_{\text{pred}} = x_{\text{true}} \end{cases}$$

- The squared error is differentiable everywhere, the MAE derivative does not exist in 0, this complicates analysis and numerical computations

Two further and important reasons

- The above are *mathematically convenient reasons*, but there are two more profound “mathematical coincidences” for which the norm-2 is a better choice...
 - When fitting a Gaussian pdf to a dataset, the **maximum likelihood fit** is the one *minimizing the squared error*, **not the absolute error**. For a precise account of this result, see, e.g., Chapter 3 “Linear models for regression” of [\[Bishop2006\]](#)
 - When doing **dimensionality reduction**, finding the basis that minimizes the norm-2 yields PCA. PCA has a natural interpretation in terms of multivariate Gaussian distribution, i.e., finding the axes of the ellipse of the pdf makes. There is a “robust PCA” variant that minimizes the MAE, but it is hard to compute and not very popular...

[\[Bishop2006\]](#) C. Bishop, “Pattern recognition and machine learning,” Springer, 2006.

APPENDIX 2 – INPUT VS OUTPUT VARIANCE OF PCA

Linear AE: property P1

Property P1: We **prove** that the covariance of the output vectors $\mathbf{y}_k$ is the best p-rank approximation of the covariance of the input vectors $\mathbf{x}_k$, $k=1,2,\dots,n$

- Since (zero mean data)

$$\mathbf{X}' = \mathbf{U}\Sigma\mathbf{V}^T$$

- For the covariance of the input vectors, it holds

$$n\text{Cov}(\mathbf{X}') = \mathbf{X}'(\mathbf{X}')^T = \mathbf{U}\Sigma\mathbf{V}^T\mathbf{V}\Sigma^T\mathbf{U}^T = \mathbf{U}\Sigma^2\mathbf{U}^T$$

- For the output covariance (from def. of covariance)

$$\begin{aligned} n\text{Cov}(\mathbf{Y}') &= (\mathbf{Y} - \bar{\mathbf{y}}\mathbf{1}^T)(\mathbf{Y} - \bar{\mathbf{y}}\mathbf{1}^T)^T = \\ &= (\mathbf{Y} - \bar{\mathbf{y}}\mathbf{1}^T)(\mathbf{Y}^T - \mathbf{1}\bar{\mathbf{y}}^T) = ? \end{aligned}$$

Proof of Property P1

- From (24)

$$\mathbf{Y} = \mathbf{W}_2 \mathbf{C} + \left(\bar{\mathbf{x}} - \frac{\mathbf{W}_2 \mathbf{C} \mathbf{1}}{n} \right) \mathbf{1}^T$$

- Using $\bar{\mathbf{y}} = \bar{\mathbf{x}}$, we have

$$\mathbf{Y} - \bar{\mathbf{y}} \mathbf{1}^T = \mathbf{Y} - \bar{\mathbf{x}} \mathbf{1}^T = \mathbf{W}_2 \mathbf{C} - \frac{\mathbf{W}_2 \mathbf{C} \mathbf{1} \mathbf{1}^T}{n} = \mathbf{W}_2 \mathbf{C}'$$

- Using

$$\mathbf{W}_2 \mathbf{C}' = \mathbf{U} \Sigma_p \mathbf{V}^T$$

$$\begin{aligned} \text{Cov}(\mathbf{Y}) &= (\mathbf{Y} - \bar{\mathbf{y}} \mathbf{1}^T)(\mathbf{Y} - \bar{\mathbf{y}} \mathbf{1}^T)^T = \mathbf{W}_2 \mathbf{C}' (\mathbf{W}_2 \mathbf{C}')^T \\ &= \mathbf{U} \Sigma_p \mathbf{V}^T \mathbf{V} \Sigma_p^T \mathbf{U}^T = \mathbf{U} \Sigma_p^2 \mathbf{U}^T \end{aligned}$$

QED

Linear AE is equivalent to PCA

- From **property P1**, this means that the linear autoencoder is an **indirect way** of performing the **Karhunen-Loève transform (PCA)** on zero average data **[Ahmed1975]**
- Hence, the linear autoencoder **applies PCA to the input data** in the sense that its code **$\mathbf{c}$** is a projection of the data into the low-dimensional principal subspace
- However, unlike PCA, the coordinates of **$\mathbf{c}$** are correlated and not necessarily sorted in descending order of variance (eigenvalues)

[Ahmed1975] N. Ahmed, K. R. Rao, “Orthogonal transforms for digital signal processing,” Springer, New York Berlin Heidelberg, 1975.

APPENDIX 3 – DERIVATIVE OF MATRIX TRACE

Autoencoder – optimal bias $\mathbf{b}_2$

$$\mathbf{b}_2^* = \operatorname{argmin}_{\mathbf{b}_2} \|\mathbf{X} - \mathbf{W}_2\mathbf{C} - \mathbf{b}_2\mathbf{1}^T\|_F^2$$

$$\|\mathbf{A}\|_F^2 = \operatorname{trace}(\mathbf{A}\mathbf{A}^T)$$

$$\nabla_{\mathbf{b}_2} (\operatorname{trace}(\mathbf{A}\mathbf{A}^T)) = \mathbf{0}$$

By direct calculation

$$\begin{aligned}\mathbf{A}\mathbf{A}^T &= (\mathbf{X} - \mathbf{W}_2\mathbf{C} - \mathbf{b}_2\mathbf{1}^T)(\mathbf{X} - \mathbf{W}_2\mathbf{C} - \mathbf{b}_2\mathbf{1}^T)^T = \\ &= (\mathbf{X} - \mathbf{W}_2\mathbf{C} - \mathbf{b}_2\mathbf{1}^T)(\mathbf{X}^T - \mathbf{C}^T\mathbf{W}_2^T - \mathbf{1}\mathbf{b}_2^T) = \\ &= \mathbf{X}\mathbf{X}^T - \mathbf{X}\mathbf{C}^T\mathbf{W}_2^T - \mathbf{X}\mathbf{1}\mathbf{b}_2^T - \mathbf{W}_2\mathbf{C}\mathbf{X}^T + \mathbf{W}_2\mathbf{C}\mathbf{C}^T\mathbf{W}_2^T + \mathbf{W}_2\mathbf{C}\mathbf{1}\mathbf{b}_2^T + \\ &\quad - \mathbf{b}_2\mathbf{1}^T\mathbf{X}^T + \mathbf{b}_2\mathbf{1}^T\mathbf{C}^T\mathbf{W}_2^T + \mathbf{b}_2\mathbf{1}^T\mathbf{1}\mathbf{b}_2\end{aligned}$$

Autoencoder – optimal bias $\mathbf{b}_2$

$$\mathbf{b}_2^* = \operatorname{argmin}_{\mathbf{b}_2} \|\mathbf{X} - \mathbf{W}_2\mathbf{C} - \mathbf{b}_2\mathbf{1}^T\|_F^2$$

$$\|\mathbf{A}\|_F^2 = \operatorname{trace}(\mathbf{A}\mathbf{A}^T)$$

$$\nabla_{\mathbf{b}_2} (\operatorname{trace}(\mathbf{A}\mathbf{A}^T)) = \mathbf{0}$$

By direct calculation

these term do not depend on $\mathbf{b}_2$
→ their contribution is zero

$$\begin{aligned} \mathbf{A}\mathbf{A}^T &= (\mathbf{X} - \mathbf{W}_2\mathbf{C} - \mathbf{b}_2\mathbf{1}^T)(\mathbf{X} - \mathbf{W}_2\mathbf{C} - \mathbf{b}_2\mathbf{1}^T)^T = \\ &= (\mathbf{X} - \mathbf{W}_2\mathbf{C} - \mathbf{b}_2\mathbf{1}^T)(\mathbf{X}^T - \mathbf{C}^T\mathbf{W}_2^T - \mathbf{1}\mathbf{b}_2^T) = \\ &= \cancel{\mathbf{X}\mathbf{X}^T} - \cancel{\mathbf{X}\mathbf{C}^T\mathbf{W}_2^T} - \cancel{\mathbf{X}\mathbf{1}\mathbf{b}_2^T} - \cancel{\mathbf{W}_2\mathbf{C}\mathbf{X}^T} + \mathbf{W}_2\mathbf{C}\mathbf{C}^T\mathbf{W}_2^T + \mathbf{W}_2\mathbf{C}\mathbf{1}\mathbf{b}_2^T + \\ &\quad - \mathbf{b}_2\mathbf{1}^T\mathbf{X}^T + \mathbf{b}_2\mathbf{1}^T\mathbf{C}^T\mathbf{W}_2^T + \mathbf{b}_2\mathbf{1}^T\mathbf{1}\mathbf{b}_2^T \end{aligned}$$

Autoencoder – optimal bias $\mathbf{b}_2$

Hence, we need to compute

$$\nabla_{\mathbf{b}_2}(\text{trace}(-\mathbf{X}\mathbf{1}\mathbf{b}_2^T + \mathbf{W}_2\mathbf{C}\mathbf{1}\mathbf{b}_2^T - \mathbf{b}_2\mathbf{1}^T\mathbf{X}^T + \mathbf{b}_2\mathbf{1}^T\mathbf{C}^T\mathbf{W}_2^T + n\mathbf{b}_2\mathbf{b}_2^T))$$

Using, for any matrices $\mathbf{B}$, $\mathbf{C}$, ...

$$\text{trace}(\mathbf{B} + \mathbf{C} + \dots) = \text{trace}(\mathbf{B}) + \text{trace}(\mathbf{C}) + \dots$$

$$\text{trace}(\mathbf{B}) = \text{trace}(\mathbf{B}^T)$$

The above gradient is equivalent to computing

$$\nabla_{\mathbf{b}_2}(-2\text{trace}(\mathbf{X}\mathbf{1}\mathbf{b}_2^T) + 2\text{trace}(\mathbf{W}_2\mathbf{C}\mathbf{1}\mathbf{b}_2^T) + \text{trace}(n\mathbf{b}_2\mathbf{b}_2^T))$$

Autoencoder – optimal bias $\mathbf{b}_2$

Now, we set:

$$\nabla_{\mathbf{b}_2}(-2\text{trace}(\mathbf{X}\mathbf{1}\mathbf{b}_2^T) + 2\text{trace}(\mathbf{W}_2\mathbf{C}\mathbf{1}\mathbf{b}_2^T) + n\text{trace}(\mathbf{b}_2\mathbf{b}_2^T)) = 0$$

and using

$$\nabla_{\mathbf{b}_2}(\text{trace}(\mathbf{b}_2\mathbf{b}_2^T)) = 2\mathbf{b}_2$$

$$\nabla_{\mathbf{b}_2}(\text{trace}(\mathbf{X}\mathbf{1}\mathbf{b}_2^T)) = \mathbf{X}\mathbf{1}$$

$$\nabla_{\mathbf{b}_2}(\text{trace}(\mathbf{W}_2\mathbf{C}\mathbf{1}\mathbf{b}_2^T)) = \mathbf{W}_2\mathbf{C}\mathbf{1}$$

We finally get

$$\mathbf{b}_2^* = \frac{\mathbf{X}\mathbf{1} - \mathbf{W}_2\mathbf{C}\mathbf{1}}{n}$$